Coaching Class Management System

A Project Report

Submitted in partial fulfillment of the
Requirements for the award of the degree

BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

By

Mr. Shesh Mangesh Bhoir

Seat No. **3008218**

Under the esteemed guidance of
Mrs. Shilpa Nayan Kawale
Assistant Professor

DEPARTMENT OF INFORMATION TECHNOLOGY
JANATA SHIKSHAN MANDAL's
Smt. Indirabai G. Kulkarni Arts, J. B. Sawant Science College &
Sau. Jankibai Dhondo Kunte Commerce College
(Affiliated to University of Mumbai)
ALIBAUG, 402 201
MAHARASHTRA
YEAR 2025-26

JANATA SHIKSHAN MANDAL's
Smt. Indirabai G. Kulkarni Arts, J. B. Sawant Science College &
Sau. Jankibai Dhondo Kunte Commerce College
(Affiliated to University of Mumbai)
ALIBAUG – MAHARASHTRA - 402 201

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project entitled,**" Coaching Class Management System "**, is bonafied work of **Mr. Shesh Mangesh Bhoir** bearing Seat No: **3008218** submitted in partial fulfillment of the requirements for the award of degree of BACHELOR OF SCIENCE in INFORMATION TECHNOLOGY from University of Mumbai.

Internal Guide **Project Coordinator**

External Examiner

Date: **College Seal**

PROFORMA FOR THE APPROVAL PROJECT PROPOSAL

(Note: All entries of the proforma of approval should be filled up with appropriate and complete information. Incomplete proforma of approval in any respect will be summarily rejected.)

PRN No. **2023016400190041** Roll no**: JI-69**

1. Name of the Student : **Mr. Shesh Mangesh Bhoir**

2. Title of the Project : **Coaching Class Management System**

3. Name of the Guide : **Mrs. Shilpa Nayan Kawale**

4. Teaching experience of the Guide: **18+ Years**

5. Is this your first submission? Yes ✓ No

Signature of the Student

Date: 5/7/2025

Signature of the Guide

Date: 5/7/2025

Signature of the Coordinator

Date: 5/7/2025

Plagiarism Checker X - Report

Originality Assessment

25%

Overall Similarity

Date: Mar 14, 2026
Matches: 3091/12364 words
Sources: 23

Remarks: High similarity detected, please make the necessary changes to improve the writing.

Verify Report:
View certificate Online

ABSTRACT

The Coaching Class Management System is a desktop-based application developed using C# and Windows Forms to streamline the administration of coaching institutes. The system provides an integrated platform to manage students, teachers, courses, batches, and subjects with ease. It enables efficient tracking of student attendance, fee payments, and performance through an interactive interface and automated reports.

Key features include student registration and listing, teacher management, batch and course allocation, attendance recording, and a fee management module with payment tracking. The system also incorporates a secure login and user management mechanism to ensure authorized access. Reports such as attendance summaries, fee receipts, and result cards are generated dynamically to support decision-making and maintain transparency.

By automating routine administrative tasks, the Coaching Class Management System reduces manual workload, minimizes errors, and improves overall efficiency. This solution is particularly beneficial for small and medium-sized coaching institutes aiming to enhance their operational effectiveness and provide better service to students.

ACKNOWLEDGEMENT

Every research endeavour, regardless of the field, requires the expert guidance and insights from knowledgeable individuals. I am immensely pleased to express my wholehearted thanks to my Project Coordinator, **Mr. Sachin S. Bhostekar**, for his exceptional advice, enthusiastic suggestions, highly valuable support and encouragement throughout this research of the project. I am also delighted to extend my heartfelt gratitude to my Project Guide, **Mrs. Shilpa N. Kawale**, for his significant contributions to the project. His dedication, expertise, and innovative approach were pivotal in driving the project to success.

I wish to convey my profound gratitude to Principal, **Dr. Mrs. Sonali Patil**, and our IT Department In-Charge, **Mr. Satyajit Tulpule**, for their sincere help and constant encouragement.

I am also thankful to our CS Department In-Charge, **Mrs. Shilpa N. Kawale**, and all the department staff for their years of experience, excellent support, and valuable suggestions.

Each of you brought unique skills and perspectives that enriched the development of my Coaching Class Management System. Your contributions have ensured not only the effectiveness and reliability of the solution but have also set a high standard for future initiatives. I anticipate continuing our collaborative journey, building upon this success, and reaching even greater achievements.

DECLARATION

I hereby declare that the project entitled, "**Coaching Class Management System"** done at place where the project is done, has not been in any case duplicated to submit to any other university for the award of any degree. To the best of my knowledge other than me, no one has submitted to any other university.

The project is done in partial fulfillment of the requirements for the award of degree of **BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)** to be submitted as final semester project as part of our curriculum.

Whenever references have been made to previous work of other, it has been clearly indicated as such and included in the bibliography and references.

(Mr. Shesh Mangesh Bhoir)

Table of Contents

Table of Figures

CHAPTER 1: INTRODUCTION

1.1 Background

Coaching institutes play a vital role in supplementing school and college education by providing specialized training and guidance to students. However, the traditional management of coaching classes often relies on manual record-keeping, which is time-consuming, error-prone, and inefficient. Maintaining separate registers for student details, teacher information, batches, subjects, attendance, and fee collection creates difficulties in data retrieval and report generation.

With the growing number of students and increasing competition in the education sector, institutes require a reliable system to handle their administrative activities. Manual systems fail to provide quick access to records, generate accurate reports, or track student progress effectively. Furthermore, monitoring fee payments and attendance manually can lead to discrepancies and financial mismanagement.

The need for an automated, computer-based solution has therefore become essential. A coaching class management system addresses these challenges by integrating various functions into a single platform. It not only reduces administrative workload but also enhances transparency, accuracy, and efficiency in managing day-to-day operations.

1.2 Objectives

- Student Management – To maintain accurate records of student details including personal information, enrolment, attendance, and performance.
- Teacher Management – To organize and manage teacher profiles, subjects handled, and batch allocations.
- Batch and Course Management – To facilitate the creation, modification, and allocation of courses, subjects, and batches.
- Attendance Tracking – To automate attendance recording and generate attendance reports for monitoring student participation.
- Fee and Payment Management – To track student fee payments, generate receipts, and manage pending dues efficiently.
- User Authentication & Security – To provide secure login and role-based access control for administrators and staff.

- Report Generation – To generate various reports such as attendance reports, fee reports, and result cards for decision-making and transparency.
- Efficiency & Accuracy – To minimize manual effort, reduce errors, and streamline the overall administration process of coaching classes.

1.3 Purpose, Scope, and Applicability

1.3.1 Purpose

The primary purpose of the Coaching Class Management System is to effectively monitor and manage student-related academic and administrative information in an organized and accessible manner. It ensures accurate tracking of student data such as attendance, results, and fee payments. The system enhances transparency by providing real-time visibility into student performance, class records, and academic status, thereby assisting administrators, teachers, and staff in making informed decisions.

In addition, the system streamlines administrative tasks such as student registration, login authentication, and report generation, thereby reducing manual workload, minimizing errors, and increasing overall efficiency. It also improves accountability by managing user roles, assigning responsibilities to administrative staff, and supporting better workflow organization. This contributes to enhanced employee productivity and improved communication across the institution.

Ultimately, the system is designed to deliver better educational services and increase student satisfaction by ensuring timely updates, efficient management, and easy access to academic records.

1.3.2 Scope

- Managing student records such as registration, contact details, batch assignment, and attendance.
- Maintaining teacher profiles, subject allocation, and teaching schedules.
- Handling fee collection, payment receipts, and pending fee reports.
- Generating reports on attendance, fees, student lists, and academic performance.
- Providing login-based authentication to ensure system security and restricted access.
- Supporting efficient data storage and retrieval to improve operational speed and accuracy.

Methodology

Methodologies are comprehensive, multiple-step approaches to systems development that guide the overall workflow and influence the quality of the final information system product (Modern Systems Analysis and Design, 2025).

In the proposed system, I used the **Iterative Development Model** to complete the SDLC phases. The iterative development model counters the limitations of the traditional waterfall model. The main idea is that the software should be developed in small, manageable increments, where each increment adds some functional capability to the system until the full system is implemented.

At each stage, extensions and design modifications can be made based on feedback and evaluation. This approach provides flexibility and allows for continuous improvement as new requirements emerge during development. An additional advantage of this model is that it allows for early testing of each module — such as the student module, teacher module, fee module, and attendance module — which ensures faster detection and correction of errors.

Furthermore, as in prototyping, the iterative model provides feedback to the client that helps determine the final system requirements and functionality. The model is particularly suitable for the Coaching Class Management System because it enables incremental development of various features, such as student registration, fee management, and attendance tracking, leading to a stable and fully functional system upon completion.

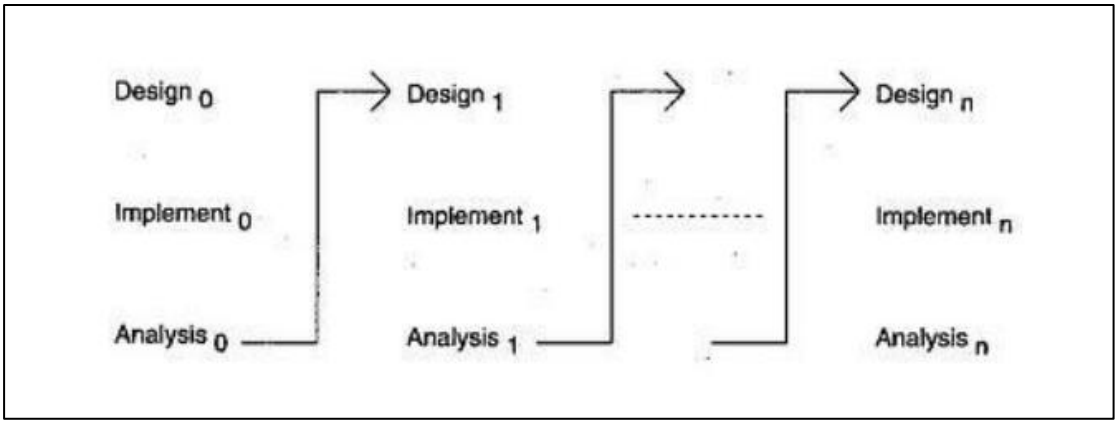


Figure 1:Iterative Enhancement Model

The **Iterative Enhancement Model** is an example of this approach. In the first step, a simple initial implementation is developed for a subset of the overall problem. This subset includes key modules that are easy to understand and implement, forming a usable foundation for the system.

A project control list is created, which contains all the tasks to be performed in order to achieve the final implementation. This list helps in tracking progress and provides insight into how far along the project is at any given step from the completed system. (Jalote)

Assumption

The system operates on a few key assumptions:

- Users (administrators, teachers, and staff) have basic knowledge of computers and can operate a Windows-based application.
- The institute maintains accurate input data such as student details, attendance, and fee records.
- The system will be used within a single coaching institute (not across multiple branches unless extended).
- Hardware and software requirements such as a Windows operating system, database, and sufficient storage are fulfilled.
- Proper internet or local network connectivity (if required for reporting or data backup) is available.
- Users will follow proper login credentials and security practices for authorized access.

Limitations

Some limitations of the Coaching Class Management System are:

- The system is designed primarily for small to medium-sized coaching institutes; scalability for very large institutions may require further enhancement.
- It is a desktop-based application (Windows Forms), which limits access to only those computers where the software is installed. Remote or mobile access is not available.
- Data security depends on the local system; without proper backups, there is a risk of data loss due to system failures.

- The system does not include advanced analytics or AI-based features for student performance prediction.
- Customization for specialized coaching centers (e.g., music, sports, or vocational training) may require modifications.
- Multi-language support is limited unless explicitly integrated.

1.3.3 Applicability

Areas of Applicability:

- **Student Information Management** – Maintains detailed student profiles, enrolment records, and academic progress tracking.
- **Course and Batch Management** – Organizes multiple courses and assigns students and faculty into specific batches.
- **Attendance Management** – Records and monitors student attendance in a systematic manner.
- **Fee Collection and Tracking** – Automates fee calculations, maintains payment records, and generates receipts.
- **Communication and Notifications** – Sends updates about class schedules, examinations, and important announcements to students.
- **Reporting and Analysis** – Generates reports related to fees, attendance, and student performance for administrative purposes.

Who Can Use It:

- Coaching classes for school and college-level subjects.
- Private tutoring centres managing multiple batches.
- Professional training institutes offering skill-development courses.

1.4 Achievements

- **Automation of Core Operations** – Replaced manual processes for student enrolment, attendance tracking, and fee collection with a fully functional digital system.
- **User-Friendly Interface** – Designed and developed an intuitive interface that allows administrators, teachers, and students to interact with the system easily.
- **Centralized Data Management** – Implemented a secure, centralized database to store and manage student records, class schedules, and fee details.

- **Modular System Architecture** – Built the system using a modular design, enabling each module (Attendance, Fees, Courses, etc.) to function independently while remaining integrated.
- **Automated Report Generation** – Developed reporting features for attendance, fee status, and student records, making administrative tasks faster and more accurate.
- **Secure Login System** – Introduced a multi-role login mechanism (admin, faculty, student) with authentication to ensure data privacy and controlled access.
- **Scalability for Future Enhancements** – Designed the system with provisions for future integration of features such as mobile app support, online payment gateways, and biometric attendance.
- **Successful Testing and Deployment** – Completed unit, integration, and user acceptance testing, followed by successful deployment in a real-time environment.

1.5 Organization of Report

This project report is structured to provide an introduction to the system, a technology analysis survey, and system design. Before diving into these sections, this overview aims to offer a comprehensive understanding of the subject. The chapters are organized as follows:

- Chapter 2: This chapter examines current technologies in the computer field, including surveys and comparative analyses relevant to project development.
- Chapter 3: This chapter outlines the system requirements specifications, covering project planning and scheduling with PERT and GANTT charts.
- Chapter 4: This chapter discusses system modularization, data design with schemas, creating data structures, designing relational schemas, developing the user interface, implementing security mechanisms, and verifying results for accuracy.
- Chapter 5: This chapter presents the overall implementation approach adopted for the project, detailing the system development process, tools, and technologies used.
- Chapter 6: This chapter provides a comprehensive testing report, including test cases, test results, and validation outcomes.

- Chapter 7: This chapter summarizes the overall system, highlighting the objectives achieved, key outcomes, and contributions of the project.

CHAPTER 2: SURVEY OF TECHNOLOGIES

In this chapter, the information about the software required to complete the project is given. Details of other technologies used are also mentioned in this chapter. Recently, many software and technologies are available to develop projects.

Front-end technologies such as React, HTML, CSS, and JavaScript are highly popular. Surveys indicate that React is among the most commonly used libraries for developing user interfaces due to its component-based structure and effective state management. HTML is essential for structuring web content, while CSS is used for styling it. JavaScript is a versatile, high-level programming language essential for building dynamic and interactive web applications. Node.js and MongoDB are widely used in back-end development today (OpenAI, 2025). Node.js, a JavaScript runtime built on Chrome's V8 engine, is praised for its event-driven, non-blocking I/O model, making it ideal for scalable and high-performance applications. MongoDB, a NoSQL database, is popular for its flexible, JSON-like data storage and its efficiency in managing large volumes of unstructured data.

I have been conducting a comparative study of the aforementioned software, technologies, and tools as follows.

1. **Languages and Frameworks:** The front end of an application is the visible part that interacts with the user. Its main purpose is to facilitate effective communication by presenting content and data in an attractive, clean, and understandable manner.
    a. **C# Language:** C# syntax is highly expressive, yet it is also simple and easy to learn. The curly-brace syntax of C# will be instantly recognizable to anyone familiar with C, C++ or Java. Developers who

know any of these languages are typically able to begin to work productively in C# within a very short time. C# syntax simplifies many of the complexities of C++ and provides powerful features such as nullable value types, enumerations, delegates, lambda expressions and direct memory access, which are not found in Java. C# supports generic methods and types, which provide increased type safety and performance, and iterators, which enable implementers of collection classes to define custom iteration behaviors that are simple to use by client code. Language-Integrated Query (LINQ) expressions make the strongly-typed query a first-class language construct. (w3resource, 2025)

b. **JAVA**: Java is a general-purpose computer programming language that is concurrent, class-based, object-oriented, and specifically designed to have as few implementation dependencies as possible. It is intended to let application developers "write once, run anywhere" (WORA), meaning that compiled Java code can run on all platforms that support Java without the need for recompilation. Java applications are typically compiled to bytecode that can run on any Java virtual machine (JVM) regardless of computer architecture. (Wikipedia, 2025)

c. **C Plus Plus**: C++ is a statically typed, compiled, general-purpose, case-sensitive, free-form programming language that supports procedural, object-oriented, and generic programming. C++ is regarded as a **middle-level** language, as it comprises a combination of both high-level and low-level language features. C++ was developed by Bjarne Stroustrup starting in 1979 at Bell Labs in Murray Hill, New Jersey, as an

enhancement to the C language and originally named C with Classes but later it was renamed C++ in 1983. C++ is a superset of C, and that virtually any legal C program is a legal C++ program. (Tutorial Point, 2025)

d. **JavaScript Language:** JavaScript is a flexible, interpreted programming language renowned for its lightweight nature and support for first-class functions. Although it is primarily recognized as the scripting language for web pages, it is also employed in various non-browser environments like Node.js. JavaScript accommodates different programming paradigms, including object-oriented, imperative, and declarative styles. It plays a key role in managing the behavior of front-end elements, making web pages more interactive and dynamic (MDN Web Docs, 2025)

e. **Framework**: The .NET Framework is a managed execution environment for Windows that provides a variety of services to its running apps. It consists of two major components: the common language runtime (CLR), which is the execution engine that handles running apps, and the .NET Framework Class Library, which provides a library of tested, reusable code that developers can call from their own apps. The services that the .NET Framework provides to running apps include the following: Memory management, A common type system, An extensive class library, Development frameworks and technologies, Language interoperability, Version compatibility, Side-by-side execution, and multi-targeting. (Microsoft, 2025)

f. **React:** React is a popular JavaScript library used for creating user interfaces, especially for single-page applications that require a dynamic and interactive user experience. Created by Facebook and supported by both Facebook and a wide developer community, React is built around a component-based structure. This allows developers to construct self-contained components that handle their own state and then combine them to form more complex UIs. React's declarative syntax makes it easier to manage and understand both application state and user interfaces. It also features JSX, a syntax extension that looks similar to HTML, which simplifies component design. A key feature of React is its virtual DOM, which boosts performance by reducing the need for direct manipulation of the real DOM. Additionally, React's hooks feature enables functional components to manage state and other functionalities that were previously limited to class-based components. With a robust ecosystem including tools like React Router and Redux, React offers a comprehensive solution for building dynamic web applications. (Flatirons, 2025)

g. **Node.js:** Node.js is a runtime environment that enables JavaScript to be run on the server side. Several frameworks built on top of Node.js simplify server-side development and add various functionalities. Express.js is a widely-used Node.js framework known for its simplicity and flexibility, often serving as a core tool for various applications. Koa.js, developed by the same team as Express, offers a more modern and streamlined approach with a focus on async functions for improved control flow. NestJS, which uses TypeScript by default, is designed for

creating scalable and maintainable server-side applications, incorporating elements from Object-Oriented, Functional, and Reactive Programming. Sails.js follows an MVC (Model-View-Controller) architecture similar to traditional frameworks but is tailored for Node.js, making it well-suited for data-driven APIs. LoopBack specializes in building and managing REST APIs and connecting them to different data sources. (OpenAI, 2025) Hapi.js is noted for its powerful plugin system and configuration-oriented development approach. Each of these frameworks has distinct advantages, so the best choice depends on the specific needs and preferences of the project.

h. **Python:** Python is a high-level, interpreted, and interactive scripting language with a focus on object-oriented programming. It's clear and simple syntax is praised for its readability, making it a great option for both newcomers and seasoned developers. Python's broad selection of libraries and frameworks adds to its versatility, enabling its use in various fields, including web development, data science, and artificial intelligence. (Python.org, 2025)

   i. **Python is Interpreted** − Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.

   ii. **Python is Interactive** − You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

   iii. **Python is Object-Oriented** − Python supports Object-Oriented style or technique of programming that encapsulates code within objects.

iv. **Python is a Beginner's Language** − Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games. (Tutorial Point, 2025)

i. **CSS**: CSS, or Cascading Style Sheets, is a stylesheet language used to manage the presentation and layout of web pages written in HTML or XML. It allows developers to define the visual appearance of web documents, including aspects like colors, fonts, spacing, and positioning. CSS operates through selectors and properties: selectors target HTML elements, while properties specify the styles to be applied. It uses the box model, which considers each element as a box with content, padding, border, and margin areas, crucial for managing layout and spacing. CSS includes various layout techniques such as Flexbox and Grid to create responsive and adaptable designs. Media queries facilitate responsive design by applying different styles based on the device's screen size or other features. The cascading nature of CSS allows multiple styles to affect the same element, with specificity rules determining which styles take precedence. Additionally, CSS can be enhanced with preprocessors like Sass and Less, and frameworks such as Bootstrap and Foundation offer pre-designed components and layout tools to speed up development. Overall, CSS plays a vital role in designing visually appealing and user-friendly web pages, greatly improving the user experience. (W3Schools, 2025)

2. **Database Management System:** The back end of application software essentially functions as the core processing unit for the front end. A database

management system (DBMS) is a type of software responsible for overseeing the storage, organization, and retrieval of data. A database application is a program designed to interact with a database, allowing users to access and manage data. Typically, a DBMS includes the following components: (Techtarget, 2025)

- Kernel code: This code manages memory and storage for the DBMS.
- Repository of metadata: This repository is usually called a data dictionary.
- Query language: This language enables applications to access the data.

I. **MongoDB:** MongoDB is a cross-platform, document-oriented database that provides, high performance, high availability, and easy scalability. MongoDB works on concept of collection and document.

   - Database is a physical container for collections. Each database gets its own set of files on the file system. A single MongoDB server typically has multiple databases.
   - Collection is a group of MongoDB documents. It is the equivalent of an RDBMS table. A collection exists within a single database. Collections do not enforce a schema. Documents within a collection can have different fields. Typically, all documents in a collection are of similar or related purpose.
   - A document is a set of key-value pairs. Documents have dynamic schema. Dynamic schema means that documents in the same collection do not need to have the same set of fields or structure, and common fields in a collection's documents may hold different types of data. (MongoDB, 2025)

II. **MySQL:** MySQL is an open-source relational database management system (RDBMS). Its name is a combination of "My", the name of co-founder Michael Widenius's daughter, and "SQL", the abbreviation for Structured Query Language. The MySQL development project has made its source code available under the terms of the GNU General Public License, as well as under a variety of proprietary agreements. MySQL was owned and sponsored by a single for-profit firm, the Swedish company MySQL AB, now owned by Oracle Corporation. For proprietary use, several paid editions are available and offer additional functionality. (MySQL, 2025)

III. **Microsoft SQL Server:** MS SQL Server is a relational database management system (RDBMS) developed by Microsoft. This product is built for the basic function of storing retrieving data as required by other applications. It can be run either on the same computer or on another across a network. (Tutorial Point, 2025)

- It is also an ORDBMS.
- It is platform dependent.
- It is both GUI and command-based software.
- It supports SQL (SEQUEL) language which is an IBM product, non-procedural, common database and case insensitive language.

IV. **PostgreSQL:** PostgreSQL is a powerful, open-source object-relational database system. It has more than 15 years of active development and a proven architecture that has earned it a strong reputation for reliability, data integrity, and correctness. PostgreSQL runs on all major operating

systems, including Linux, UNIX (AIX, BSD, HP-UX, SGI IRIX, Mac OS X, Solaris, Tru64), and Windows. This tutorial will give you quick start with PostgreSQL and make you comfortable with PostgreSQL programming. (Tutorial Point, 2025)

V. **Oracle:** Oracle Database (commonly referred to as Oracle RDBMS or simply as Oracle) is a multi-model database management system produced and marketed by Oracle Corporation. It is a database commonly used for running online transaction processing (OLTP), data warehousing (DW) and mixed (OLTP & DW) database workloads. The latest generation, Oracle Database 18c, is available on-prem, on-Cloud, or in a hybrid-Cloud environment. (Oracle Database, 2025)

VI. **Express Server:** An Express server is a lightweight and flexible web application framework for Node.js, designed to simplify the development of web and mobile applications. By using Express, developers can easily create a server that handles routing, middleware, and other server-side operations with minimal setup. To get started, you need to have Node.js installed and initialize a new Node.js project. After installing Express via npm, you can create a simple server by requiring Express in your JavaScript file, setting up routes, and defining how the server should respond to requests (OpenAI, 2025). Express also provides powerful features for routing, middleware integration, and error handling, making it a go-to framework for building robust web applications with Node.js.

Other Tools:

Apart from the main technologies, the back-end developer will also use supporting tools such as UML Modelling Software (for system design diagrams), Reporting Tools (for generating student reports, fee receipts, and attendance records), and Testing Tools (for ensuring system quality and bug-free deployment).

The following software and tools will be used to complete the proposed Coaching Class Management System project after studying and researching different software, frameworks, and tools.

I am choosing C#.NET framework for the front-end and MS SQL Server for the back-end database of the proposed project because C#.NET provides rich features and better integration with Windows applications, which is suitable for developing a robust and secure management system.

Some advantages of C#.NET for this project are:

1. **Object-Oriented and Secure** – C# is an elegant and type-safe object-oriented language that enables developers to build secure and robust applications for managing student data, class schedules, and fee structures.
2. **Ease of Use and Expressiveness** – The syntax of C# is simple, expressive, and easy to learn, which helps in faster development of forms such as student registration, attendance management, and report generation.
3. **Efficient Build Process** – The C# build process is simple compared to C and C++, and more flexible than Java, making it easier to maintain and extend the Coaching Class Management System in the future.

CHAPTER 3: REQUIREMENTS AND ANALYSIS

3.1 Problem Definition

The main aim of the proposed project is to computerize the daily transactions, billing, and report generation of the coaching class system. The existing manual system requires a lot of paperwork, time, and effort, which often leads to inconvenience and inefficiency in class management.

The proposed software package will provide a faster, more accurate, and user-friendly solution for managing student records, fee payments, attendance, and reports. It will ensure convenience for both the management and the students by simplifying operations, reducing manual labour, and improving overall efficiency.

3.3 Planning and Scheduling

The development of the Coaching Class Management System is planned in several stages to ensure smooth progress and timely completion:

1. **Define Objectives**

   Establish the goals of the project, such as automating student registration, attendance, fee management, scheduling, and report generation.

2. **Identify Target Users**

   The primary users will be coaching class administrators, faculty members, and students.

3. **Feature Selection**

   Finalize core features like student registration, attendance tracking, fee collection, timetable management, report generation, and billing.

4. **UI/UX Design**

   Prepare wireframes and simple prototypes to visualize the flow of forms, dashboards, and user interactions.

5. **Development Phase**

- Frontend: C#.NET for building user-friendly forms and interfaces.

- Backend: MS SQL Server for secure data storage and management.
- Integration: Linking the database with application modules for smooth functionality.

6. **Testing**

   Perform unit testing and integration testing to ensure all modules work correctly. Collect feedback from sample users (class staff/students).

7. **Implementation**

   Deploy the system for actual use in the coaching class, replacing the manual system.

8. **Post-Implementation Support**

   Provide regular updates, fix bugs, and add new features based on user needs.

9. **Scalability**

   Ensure the system can be extended in the future to support multiple branches, online student portals, and mobile applications.

Program Evaluation Review Technique (PERT): -

PERT charts will be used to represent the project schedule. Each task (such as design, development, testing, and implementation) will be shown as a node, with arrows indicating dependencies between tasks. Time estimates for each task will help identify the critical path—the sequence of tasks that determines the minimum time required for the project's completion. This ensures proper planning and realistic scheduling of the Coaching Class Management System.

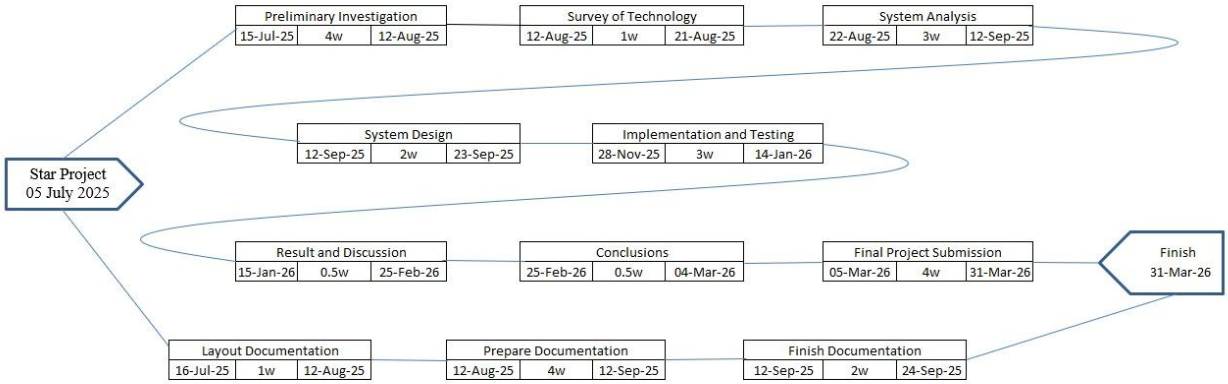


Figure 2:PERT Chart

Gantt Charts: -

Gantt charts will be used to show the timeline of each task in the project. The horizontal axis will represent the calendar duration, and the vertical axis will list the project tasks in order. Progress can be tracked by shading task boxes as they are completed. This makes it easy for the project manager to monitor the current status, delays, and dependencies between tasks.

Gantt Chart Data

ID	SDLC Phases	Start Date	Expected Date	Finish Date	Duration(days)	Extra Days
1	Introduction	15-Jul-25	12-Aug-25	12-Aug-25	29	0
2	Survey of Technology	12-Aug-25	21-Aug-25	21-Aug-25	9	0
3	Requirement and Analysis	22-Aug-25	12-Sep-25	13-Sep-25	22	1
4	System Design	13-Sep-25	23-Sep-25	26-09-2025	12	3
5	Project Report Submission	26-Sep-25	30-Sep-25	04-10-2025	8	4
6	Implementation and Testing	28-Nov-25	14-Jan-26	14-01-2026	47	0
7	Result and Discussions	15-Jan-26	11-Feb-26	25-02-2026	27	14
8	Conclusions	25-Feb-26	10-Mar-26	04-03-2026	13	0
9	Final Project Report Submission	05-Mar-26	31-Mar-26	31-03-2026	27	0

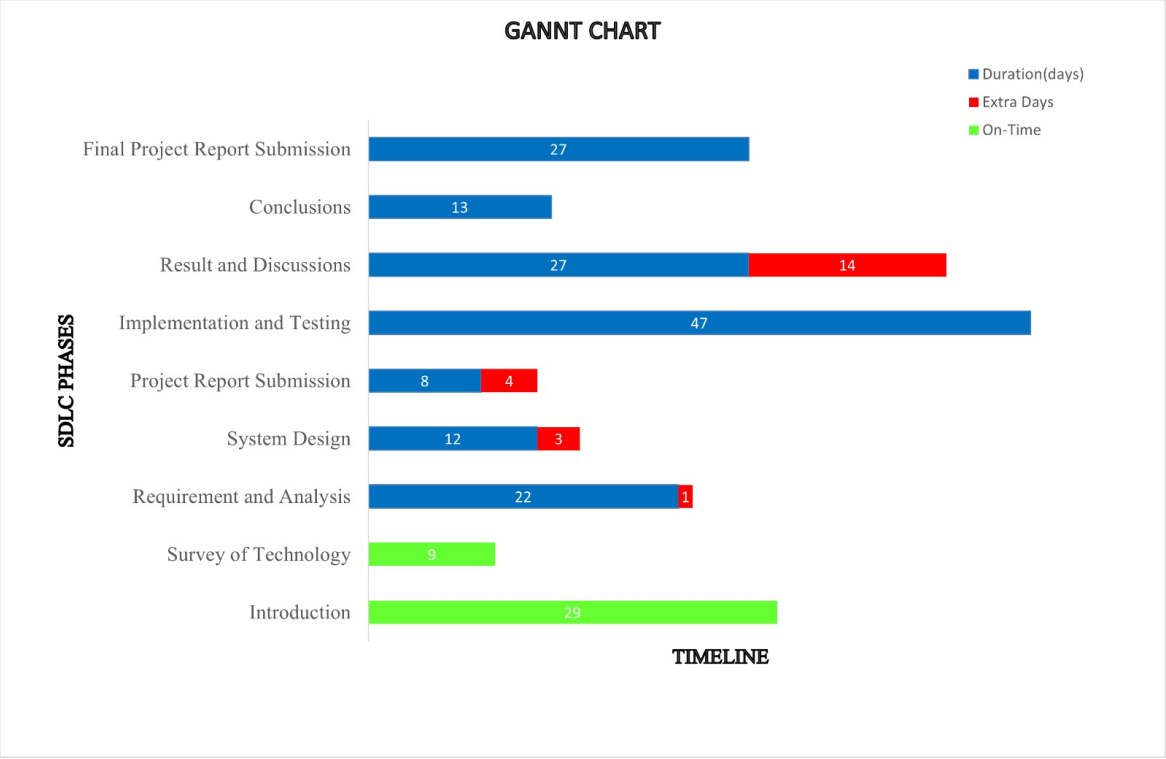


Figure 3: Gant Chart

Network Diagram:

The network diagram is based on the tasks and their dependencies. Task A has no predecessor, and therefore starts the project on the left. Task B has task A as a predecessor, while tasks D and E have tasks C as a predecessor, allowing simultaneous carrying.

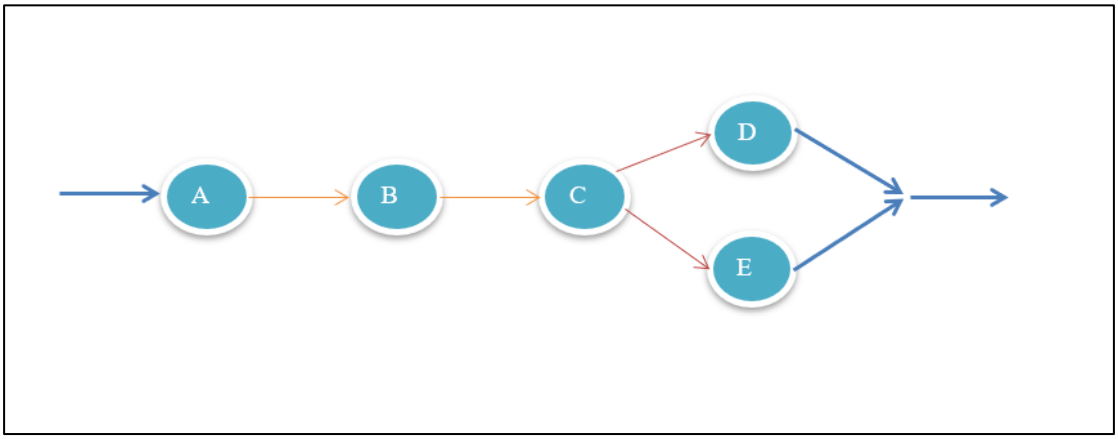


Figure 4: Network Diagram

3.4 Software and Hardware Requirements

Hardware Requirements:

- Server: High-performance server.
- Processor: 12th Gen Intel(R) Core (TM) i3-1215U @1.20 GHz
- Backup System: Reliable backup.
- Network: High-speed internet connection with low latency.
- CPU: Dual-core or higher for handling High Traffic.
- RAM: 8GB or above.
- Storage: 250 GB SSD preferred.
- Other Considerations: Scalability, Security.

Software Requirements:

Software	Functionality	Use in Project
System: Windows 10/11 or Linux	Operating System	Provides the base environment for developing and running the project.
ASP.NET with C#	Front-End Development Framework	Used to design and implement the user interface and handle client-side functionality.
MySQL	Database Management System	Used to create and manage databases, store and retrieve student, teacher, course, and fee details.
Visual Studio Code	Integrated Development Environment	Used for writing, debugging, and executing ASP.NET with C# and database integration.
StarUML	UML Modelling Tool	Used to design UML diagrams (ER, Use Case, Sequence, Activity, State, Component, Deployment).
Microsoft Word	Documentation Tool	Used to prepare project reports and documentation.
Grammarly	Grammar Checking Tool	Helps in checking and correcting grammatical mistakes in project documentation.
Plagiarism Checker	Plagiarism Checking Tool	Ensures originality and authenticity of project documentation.

3.5 Preliminary Product Description

1. Technical Feasibility

It raises questions like: Is it possible that the work can be done with the current equipment, software technology, personnel, and whether new technology is required? What is the possibility that it can be developed?

In our project, the software that we have built fully supports the current Windows operating system. It is not dependent on the number of users, so it can handle a very large number of environments.

2. Economic Feasibility

It deals with the economic impact of the system on the environment in which it is used. This includes the cost of creating this system and the benefits expected from it. A cost-benefit calculation is carried out to determine the expected benefits.

We are assuming an economically feasible solution, i.e., our project has a positive cost-benefit ratio. We also know the required amount to develop this system, and the expected benefits make it clear that the system is economically feasible. Therefore, there are no financial problems with this system.

3. Schedule Feasibility

It raises the question: Is it possible to complete the project within the given schedule, and is the schedule appropriate for the system or not? The project must be completed within the schedule, and if more advanced features are added, the schedule may expand. By developing a plan for future upgrades of the system, this impact can be minimized.

In our project, the schedule can be met because it is well-planned. Every phase of completing the project has sufficient time to finish; therefore, the project will be completed within the expected schedule.

4. Operational Feasibility

For an operationally feasible project, support for changes must be offered to maintain morale and commitment. The new system should be easy to operate for the customer and should be easily accessible.

In our project, the system will support changes that may be required. The simple user interface allows users to operate the system easily, and the system is accessible to all users. Therefore, this project is operationally feasible.

3.6 Conceptual Models

A conceptual model is a representation of a system, made of the composition of concepts which are used to help people know, understand, or simulate a subject the model represents. (OpenAI, 2025)

Entity Relationship Diagram:

An Entity Relationship (ER) Diagram is a type of flowchart that illustrates how "entities" such as people, objects or concepts relate to each other within a system. ER Diagrams are most often used to design or debug relational databases in the fields of software engineering, business information systems, education and research. Also known as ERDs or ER Models, they use a defined set of symbols such as rectangles, diamonds, ovals and connecting lines to depict the interconnectedness of entities, relationships and their attributes (Lucidchart, 2025).

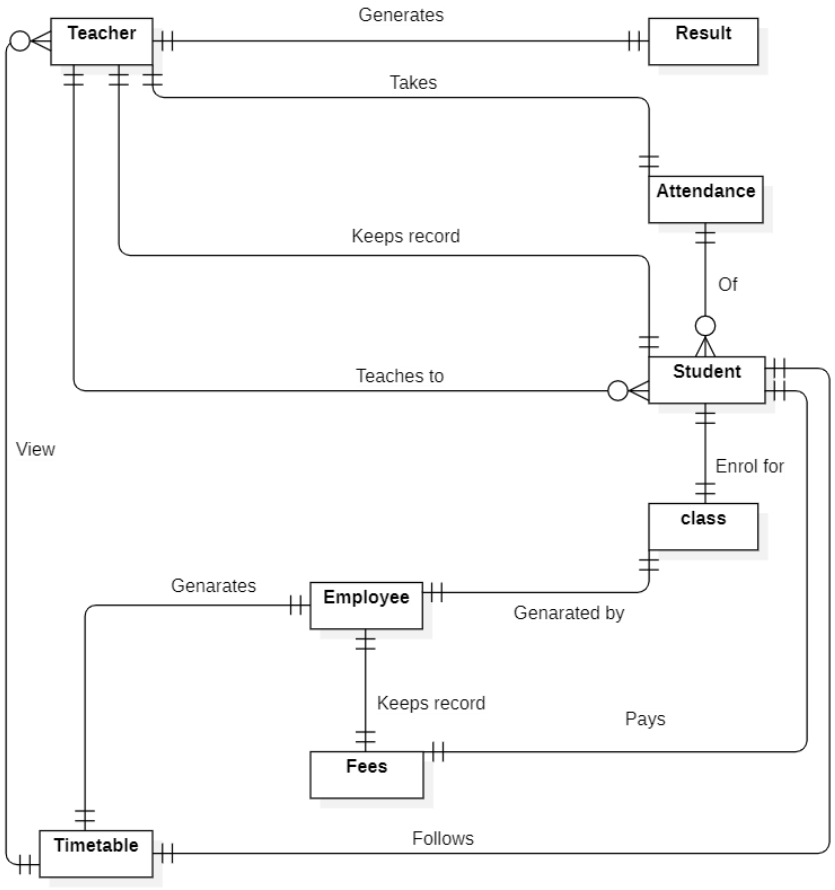


Figure 5:Entity Relationship Diagram

Class Diagram

Class diagrams are a type of UML (Unified Modeling Language) diagram used in software engineering to visually represent the structure and relationships of classes within a system i.e. used to construct and visualize object-oriented systems.

Class diagrams provide a high-level overview of a system's design, helping to communicate and document the structure of the software. They are a fundamental tool in object-oriented design and play a crucial role in the software development lifecycle. (Class Diagram | Unified Modeling Language (UML), 2025)

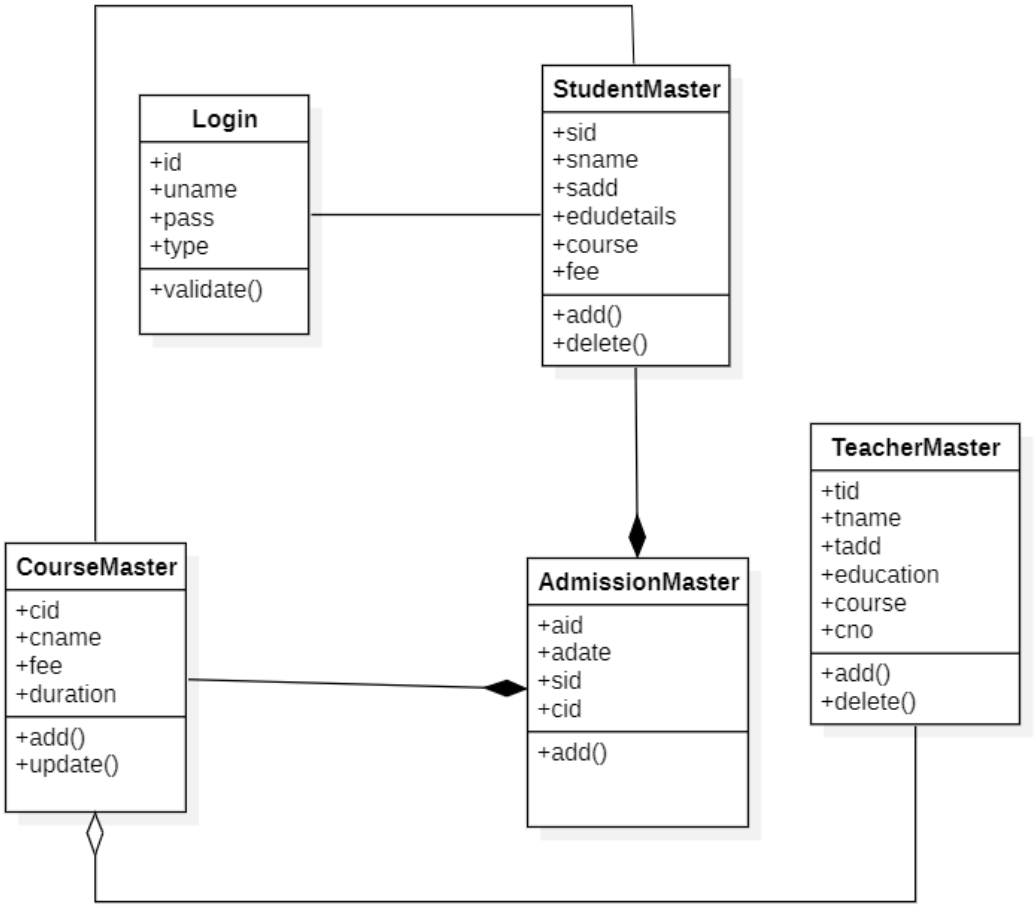


Figure 6: Class Diagram

Use Case Diagram

A **use case diagram** is a graphical depiction of a user's possible interactions with a system. A use case diagram shows various use cases and different types of users the system has and will often be accompanied by other types of diagrams as well. The

use cases are represented by either circles or ellipses. The actors are often shown as stick figures. (Wikipedia, 2025)

Figure 7: Use Case Diagram

Sequence Diagram

A sequence diagram is a Unified Modelling Language (UML) diagram that illustrates the sequence of messages between objects in an interaction. A sequence diagram consists of a group of objects that are represented by lifelines, and the messages that they exchange over time during the interaction.

A sequence diagram shows the sequence of messages passed between objects. Sequence diagrams can also show the control structures between objects. For

example, lifelines in a sequence diagram for a banking scenario can represent a customer, bank teller, or bank manager. The communication between the customer, teller, and manager are represented by messages passed between them. The sequence diagram shows the objects and the messages between the objects. (IBM, 2025)

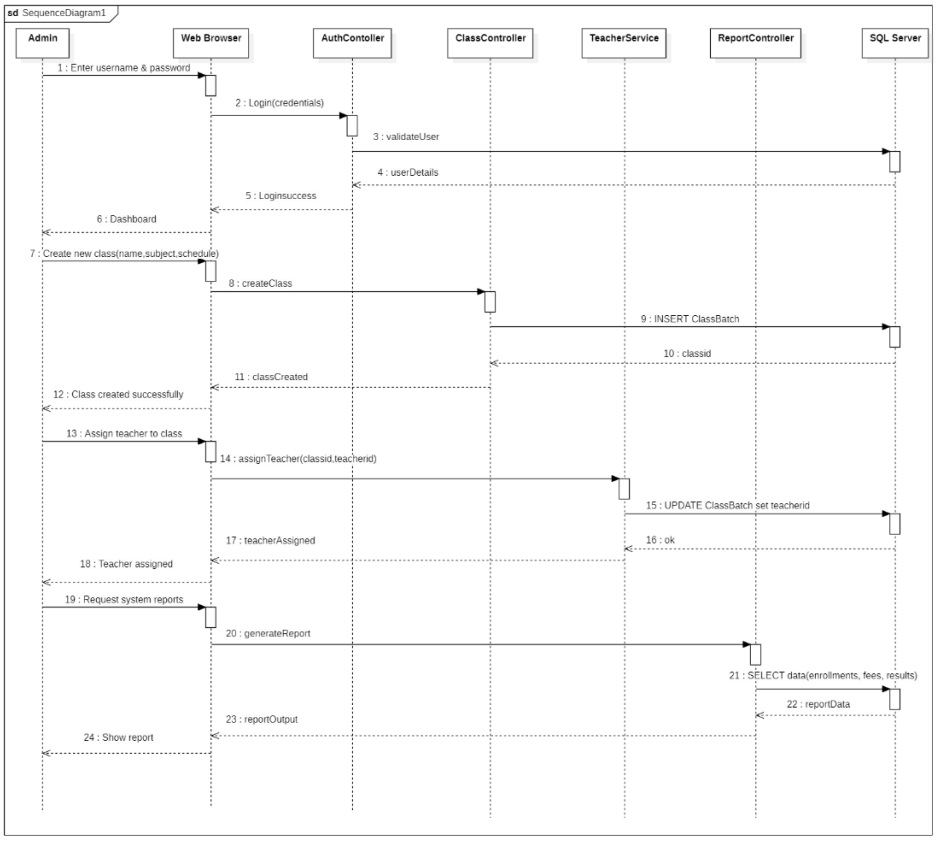


Figure 8:Sequence Diagram (Admin)

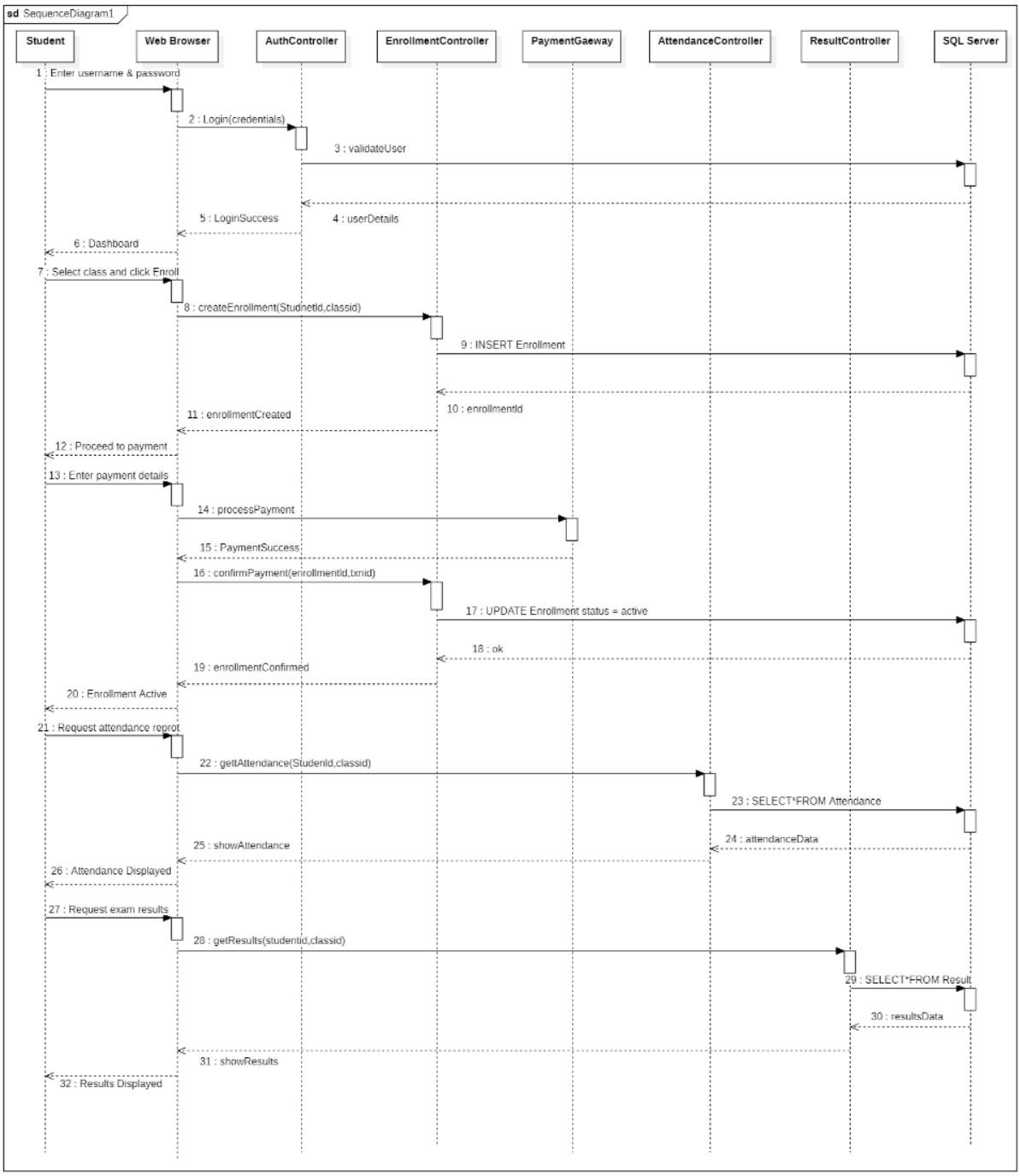


Figure 9: Sequence Diagram (Student)

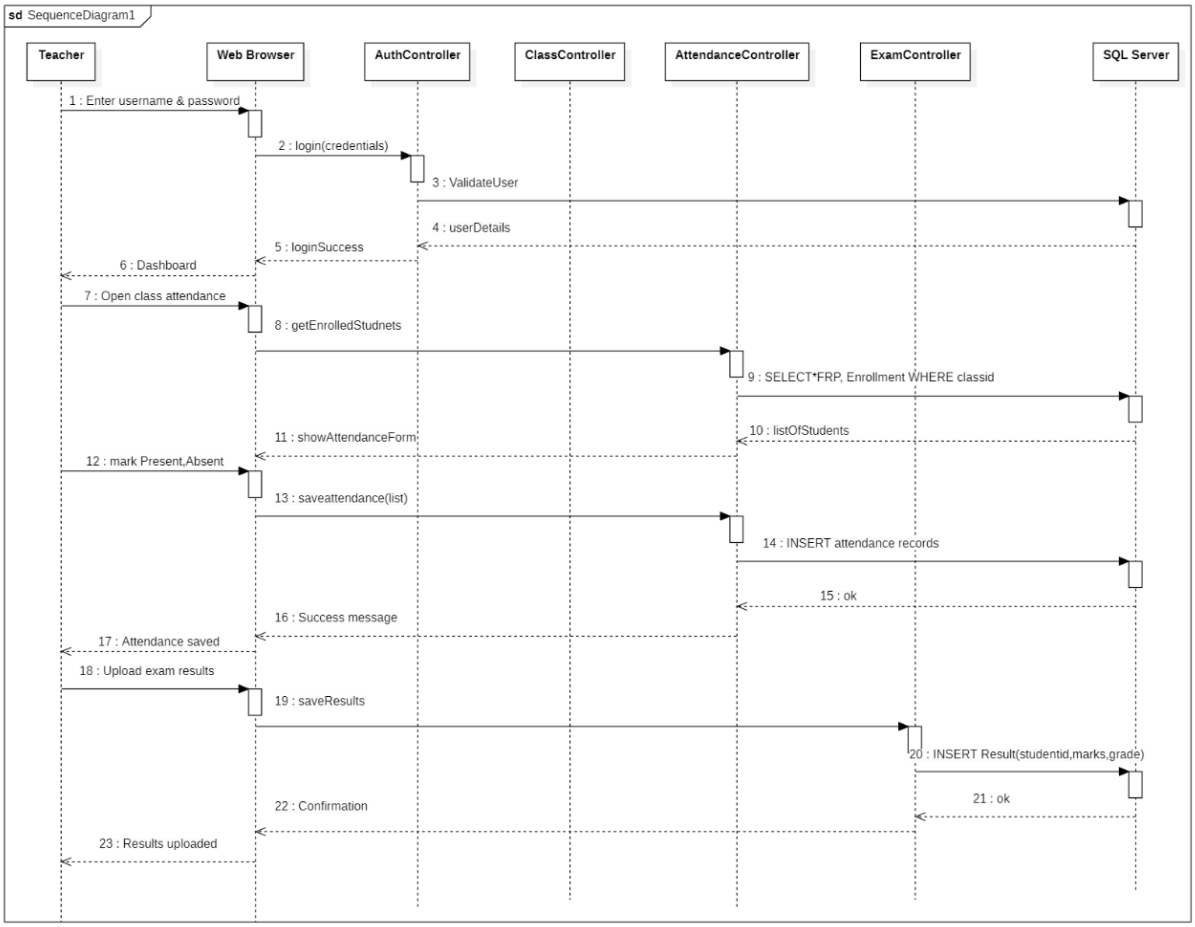


Figure 10: Sequence Diagram (Teacher)

State Chart Diagram

The name of the diagram itself clarifies the purpose of the diagram and other details. It describes different states of a component in a system. The states are specific to a component/object of a system. A State chart diagram describes a state machine. State machine can be defined as a machine which defines different states of an object and these states are controlled by external or internal events. (Tutorialspoint, 2025)

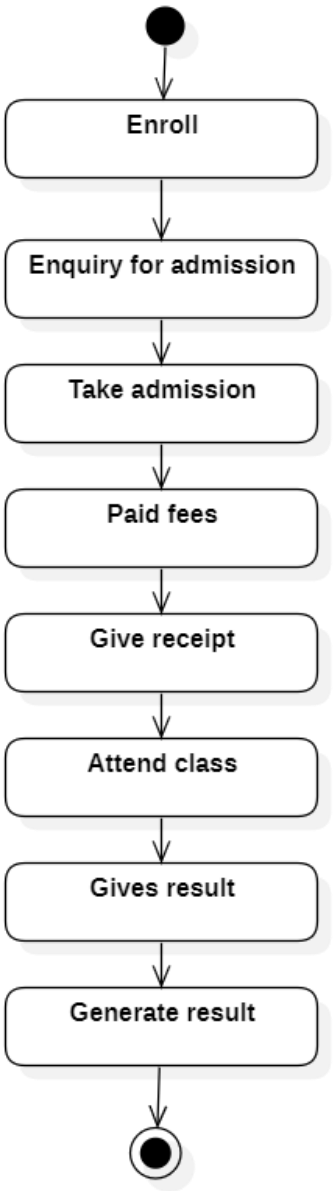


Figure 11: State Chart Diagram

Activity Diagram

An activity diagram is a type of Unified Modelling Language (UML) flowchart that shows the flow from one activity to another in a system or process. It's used to describe the different dynamic aspects of a system and is referred to as a 'behaviour diagram' because it describes what should happen in the modelled system.

Even very complex systems can be visualized by activity diagrams. As a result, activity diagrams are often used in business process modelling or to describe the steps of a use case diagram within organizations. They show the individual steps in an activity and the order in which they are presented. They can also show the flow of data between activities.

Activity diagrams show the process from the start (the initial state) to the end (the final state). Each activity diagram includes an action, decision node, control flows, start node, and end node (MindManager, 2025).

Admin:

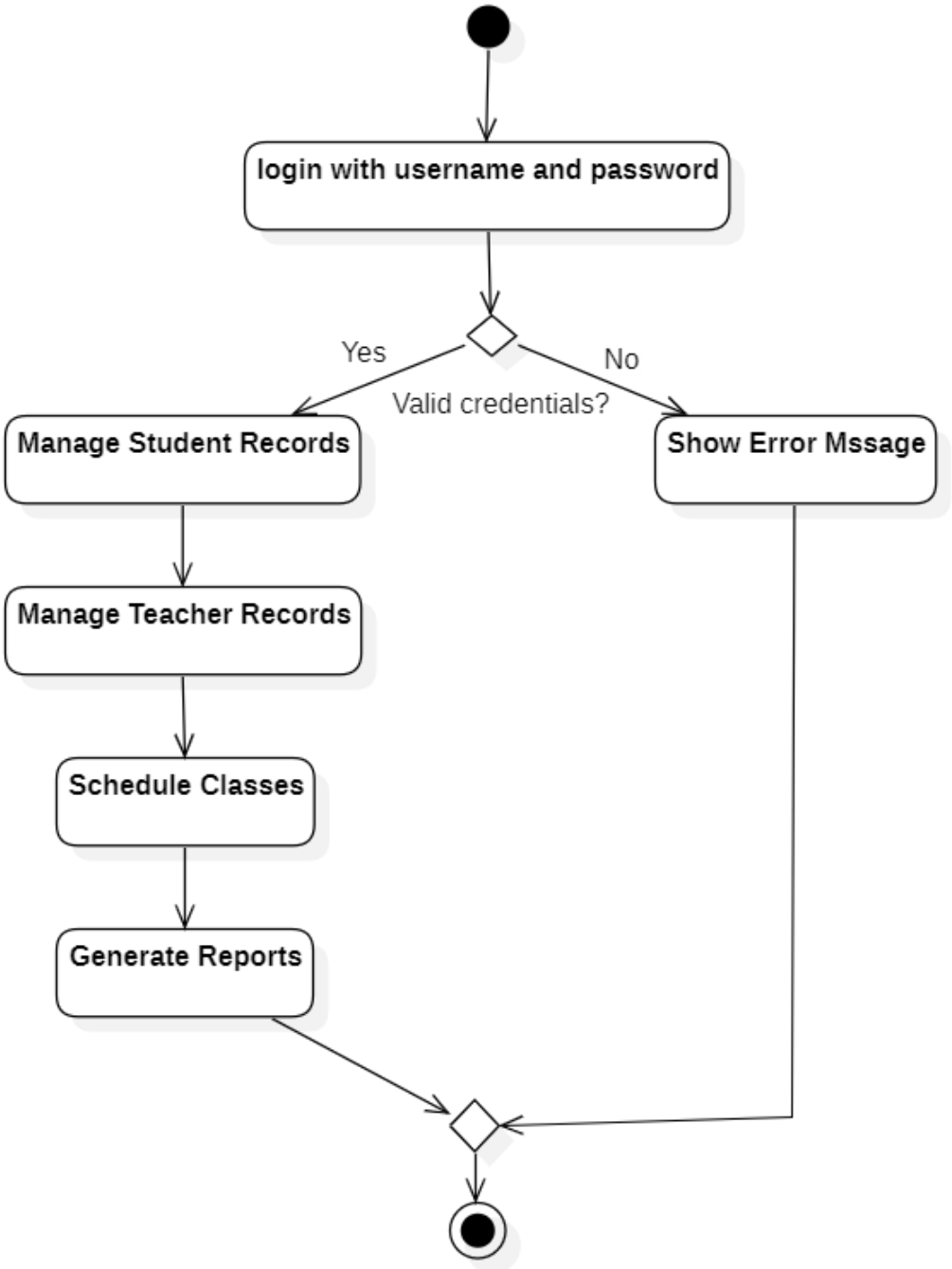


Figure 12: Activity Diagram (Admin)

Student:

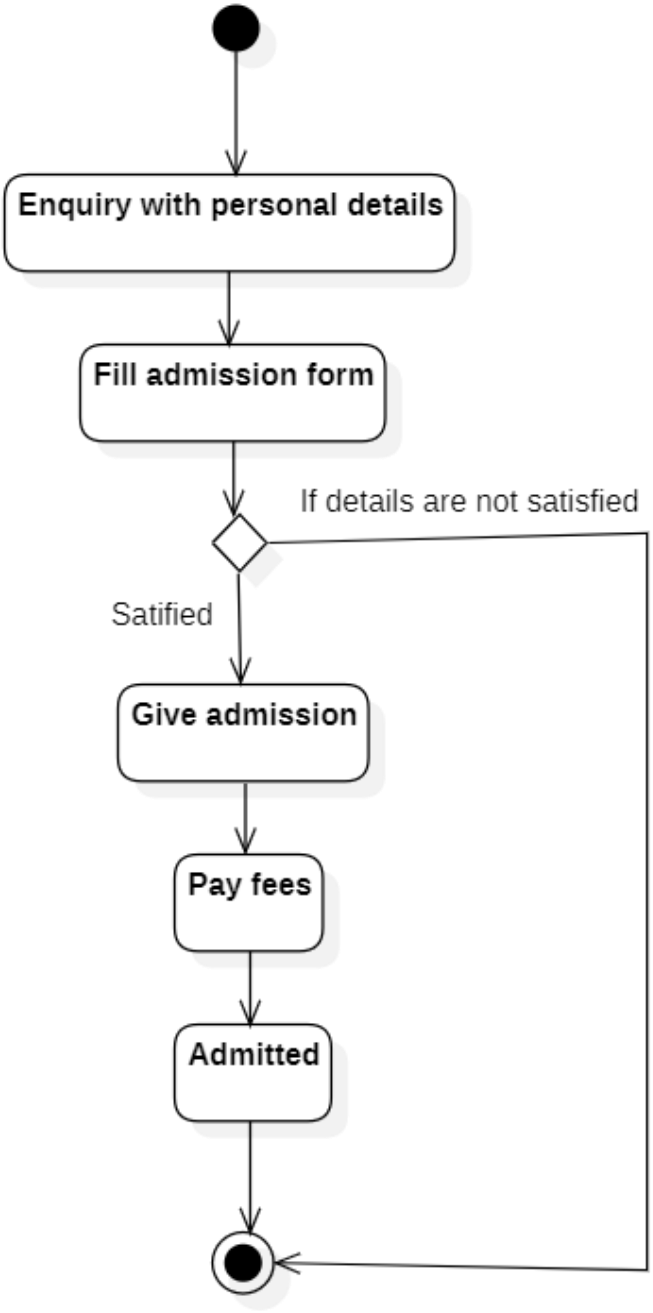


Figure 13: Activity Diagram (Student)

Teacher:

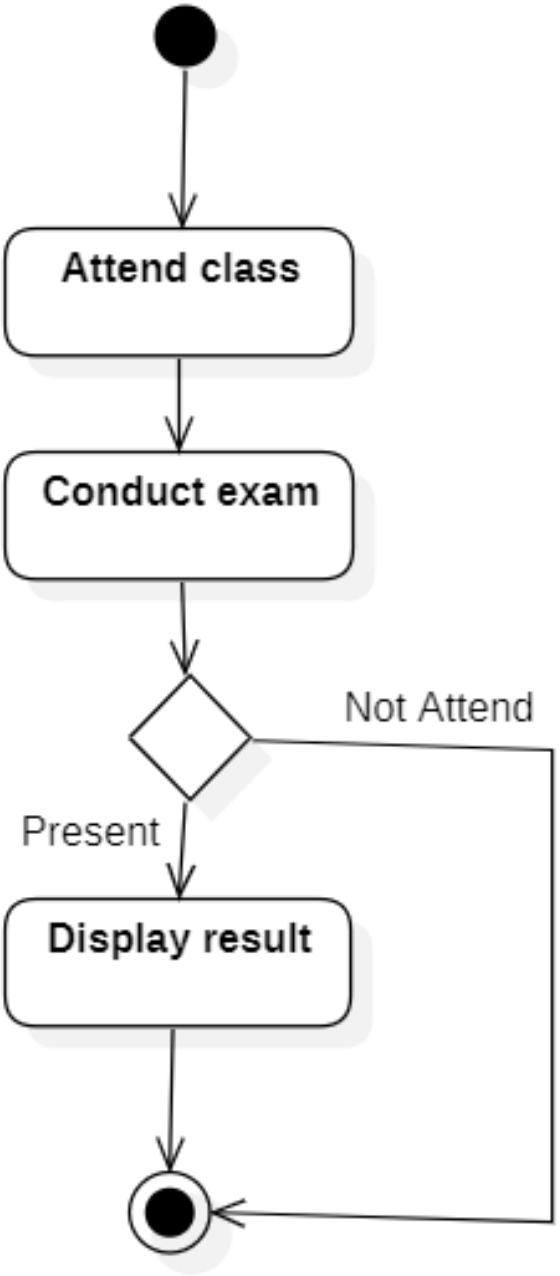


Figure 14: Activity Diagram (Teacher)

Component Diagram

A component diagram, also known as a UML component diagram, describes the organization and wiring of the physical components in a system. Component diagrams are often drawn to help model implementation details and double-check that every aspect of the system's required functions is covered by planned development.

1. **Component**

A component is a logical unit block of the system, a slightly higher abstraction than classes. It is represented as a rectangle with a smaller rectangle in the upper right corner with tabs or the word written above the name of the component to help distinguish it from a class.

2. **Interface**

An interface (small circle or semi-circle on a stick) describes a group of operations used (required) or created (provided) by components. A full circle represents an interface created or provided by the component. A semi-circle represents a required interface, like a person's input.

3. **Dependencies**

Draw dependencies among components using dashed arrows (smartdraw, 2025).

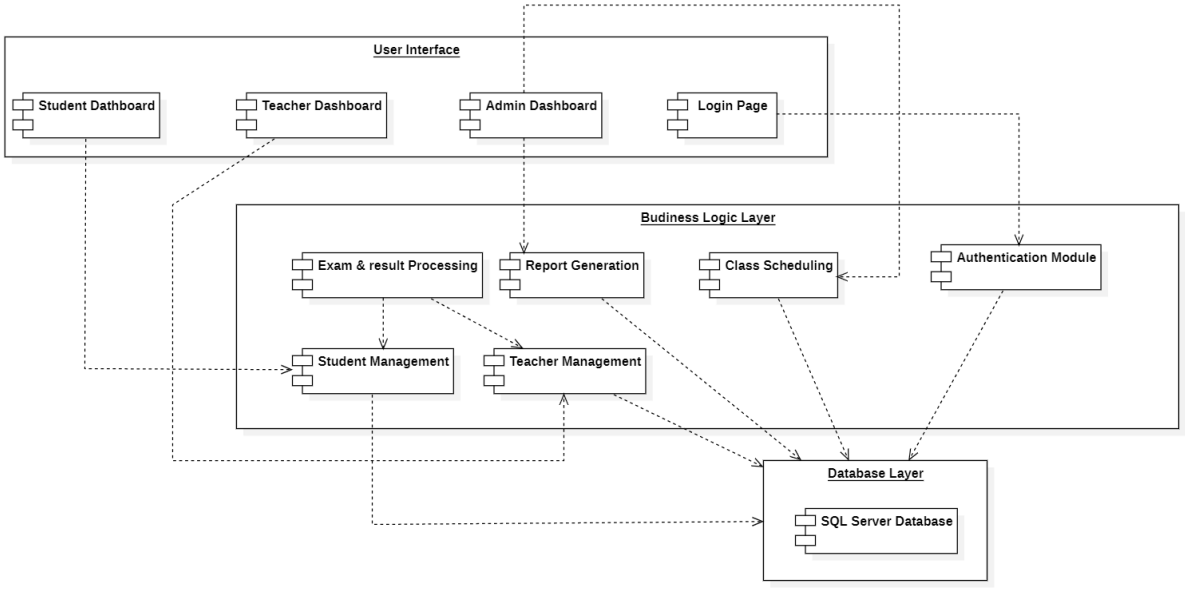


Figure 15: Component Diagram

Deployment Diagram

In UML, deployment diagrams model the physical architecture of a system. Deployment diagrams show the relationships between the software and hardware components in the system and the physical distribution of the processing.

Deployment diagrams, which you typically prepare during the implementation phase of development, show the physical arrangement of the nodes in a distributed system, the artifacts that are stored on each node, and the components and other elements that the artifacts implement. Nodes represent hardware devices such as computers, sensors, and printers, as well as other devices that support the runtime environment of a system. Communication paths and deploy relationships model the connections in the system (IBM, 2025).

Figure 16: Deployment Diagram

CHAPTER 4: SYSTEM DESIGN

4.1 Basic Modules

1. **User Authentication Module**
    - Handles sign-up and login using email/phone number.
    - Manages user profiles including student name, parent details, batch, enrolled courses, and payment status.
2. **Course & Batch Management Module**
    - Allows creation and scheduling of multiple courses (e.g., Math, Science, English).
    - Facilitates batch allocation of students and faculty.
    - Supports rescheduling, merging, or splitting batches.
3. **Student Information Module**
    - Stores complete student profiles in a database.
    - Tracks academic progress, attendance, and performance reports.
    - Maintains parent/guardian contact details.
4. **Attendance Management Module**
    - Records daily attendance digitally.
    - Provides reports of attendance percentage per student and per batch.
    - Sends automated notifications to parents for absentees.
5. **Fee Management Module**
    - Automates fee calculation based on course and batch.
    - Tracks payments (paid, pending, overdue).
    - Generates receipts and reminders for due payments.
6. **Exam & Assignment Module**
    - Allows teachers to upload assignments, quizzes, and tests.
    - Records marks/grades and maintains a performance database.
    - Generates student progress reports for teachers and parents.
7. **Communication & Notification Module**
    - Sends important updates (exam dates, batch timings, fee reminders) via SMS/email/app notifications.
    - Supports announcements at batch/course level.

8. **Learning Resources Module**
    - Provides access to notes, recorded lectures, and study material.
    - Allows uploading of PDFs, videos, and practice sheets.
    - Encourages self-paced learning beyond classroom hours.
9. **History & Review Module**
    - Stores past test records, assignments, and student reports.
    - Enables teachers and parents to review previous performance for progress tracking.
10. **Rewards & Recognition Module**
    - Tracks student achievements and awards certificates for top performance.
    - Maintains leaderboards for motivation and recognition.

4.2 Data Design

Data design is a fundamental aspect of system design that focuses on structuring, organizing, and managing data within a system. It involves several key elements, including schema design, data integrity, and constraints. Schema design defines the structure of the database, specifying tables, columns, relationships, and constraints to ensure efficient storage and retrieval of data. Data integrity ensures that data remains accurate, consistent, and reliable throughout its lifecycle by applying rules and constraints that prevent errors and maintain quality. Constraints, such as primary keys and foreign keys, enforce validity and consistency, safeguarding the data from anomalies and duplication. (Integrate.io, 2025)

Why Data Design is Important?

The importance of data design cannot be overstated. A well-designed data schema enhances efficiency by optimizing data retrieval and storage, leading to better overall system performance. Consistency is maintained through effective data design, preventing issues like data duplication and errors. Proper data design also supports scalability, allowing the system to handle growth in data volume without compromising performance.

Moreover, it simplifies system maintenance by providing a clear data structure, making updates and modifications more manageable. Security is another critical aspect, as good data design incorporates measures to protect sensitive information through access controls and encryption. Finally, effective data design facilitates better data management practices, including backup, recovery, and analysis, which are essential for deriving insights and making informed decisions. In summary, data design is crucial for creating a robust, efficient, and reliable system that meets both current and future data management needs (miro, 2025).

4.2.1 Schema Designs

- **Student**

student_id	name	email	phone	address	dob

- **Teacher**

teacher_id	name	email	phone	subject

- **Course**

course_id	course_name	duration	fees

- **Subject**

subject_id	subject_name	description	course_id

- **Batch**

subject_id	subject_name	description	course_id

- **Fee**

fee_id	student_id	course_id	amount	date_paid	mode_of_payment

4.2.2 Data Integrity and Constraints

1. **Course Table**

Column Name	Data Type	Allow Nulls
cid	nchar(10)	No
cname	nvarchar (50)	Yes
dur	numeric (18,0)	Yes
batch	nvarchar (50)	Yes
cfee	numeric (18,0)	Yes
dis	numeric (18,0)	Yes
stax	numeric (18,0)	Yes
othchg	numeric (18,0)	Yes
tfee	numeric (18,0)	Yes

2. **Batch Table**

Column Name	Data Type	Allow Nulls
bid	nchar (10)	No
bname	nvarchar (50)	Yes
sdt	date	Yes
edt	date	Yes
stime	nvarchar (50)	Yes
etime	nvarchar (50)	Yes
course	nvarchar (50)	Yes

3. Student Table

Column Name	Data Type	Allow Nulls
tid	nchar (10)	No
tname	varchar (50)	Yes
tadd	nvarchar (50)	Yes
tgen	nvarchar (50)	Yes
tqual	nvarchar (50)	Yes
tmob	nvarchar (50)	Yes
tdt	date	Yes
tsub	nvarchar (50)	Yes
tco	nvarchar (50)	Yes
tbat	nvarchar (50)	Yes

4. Attend (Attendance) Table

Column Name	Data Type	Allow Nulls
sid	nchar (10)	No
sname	nvarchar (50)	Yes
sco	nvarchar (50)	Yes
spa	nvarchar (50)	Yes
fdt	date	Yes
tdt	date	Yes
app	nvarchar (50)	Yes
res	nvarchar (50)	Yes

5. Rcard (Result Card) Table

Column Name	Data Type	Allow Nulls
sid	nchar (10)	No
sname	nvarchar (50)	Yes
cou	nvarchar (50)	Yes
sub	nvarchar (50)	Yes
ms	numeric (13,0)	Yes
tm	numeric (18,0)	Yes
pm	numeric (18,0)	Yes
edt	date	Yes

6. Fee Table

Column Name	Data Type	Allow Nulls
tid	nchar(10)	No
tname	nvarchar (50)	Yes
tco	nvarchar (50)	Yes
tsub	nvarchar (50)	Yes
pamt	numeric (18,0)	Yes
pdt	date	Yes
pmon	nvarchar (50)	Yes

4.3 Procedural Design

The procedural design is often understood as software design; The procedural design includes following topics.

4.3.1 Logic Diagrams

Process diagram

An initial diagram is a general overview of what the stakeholders think the business process looks like.

Admin:

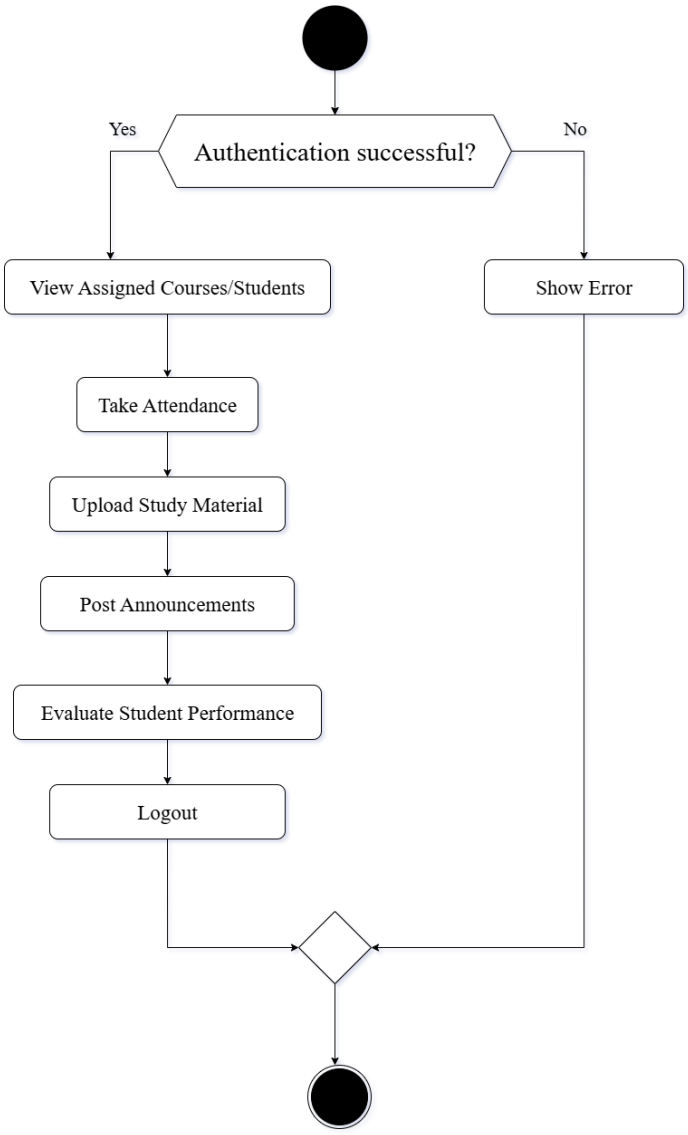


Figure 17: Process Diagram (Admin)

Student:

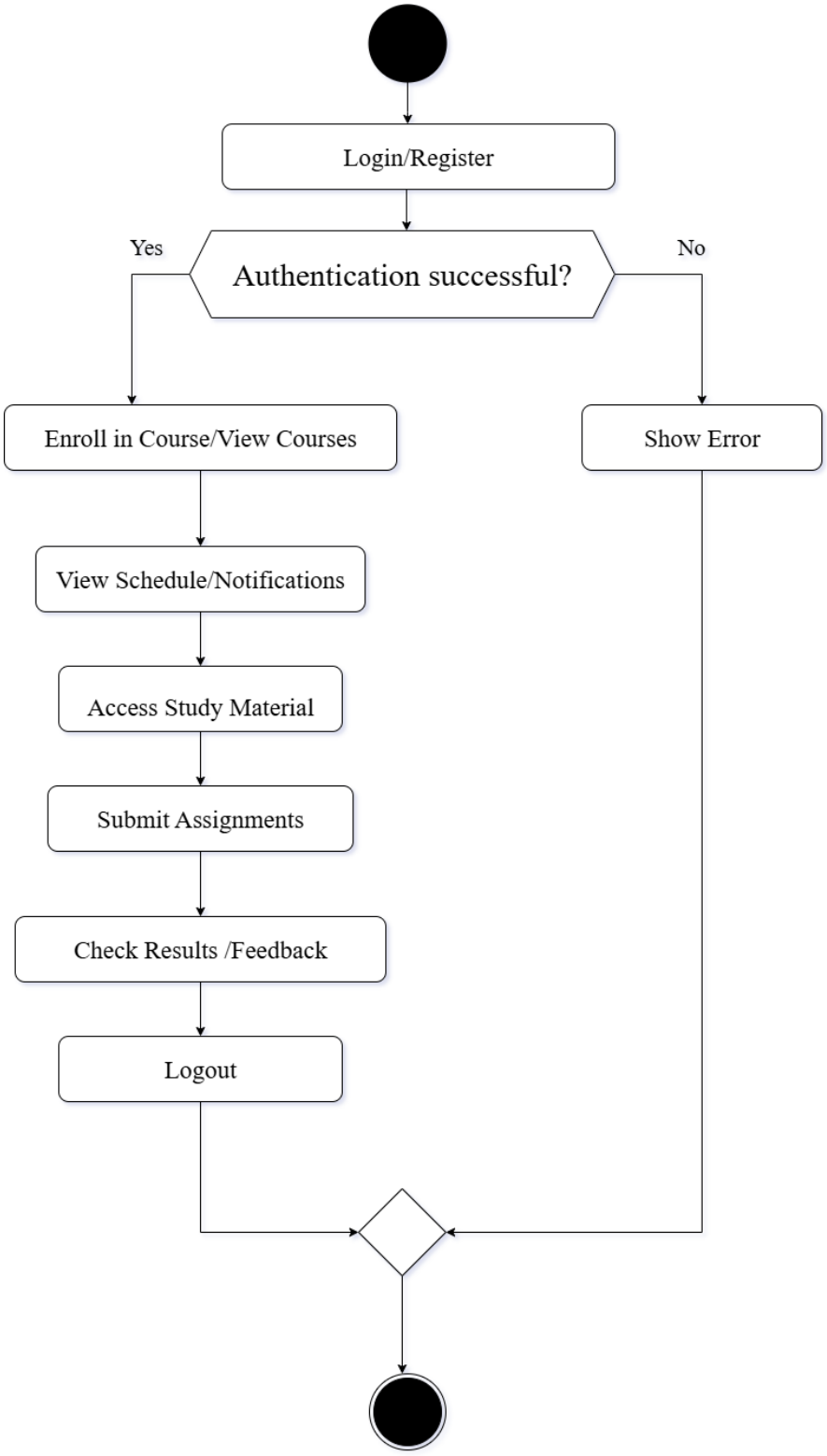


Figure 18: Process Diagram (Student)

Teacher:

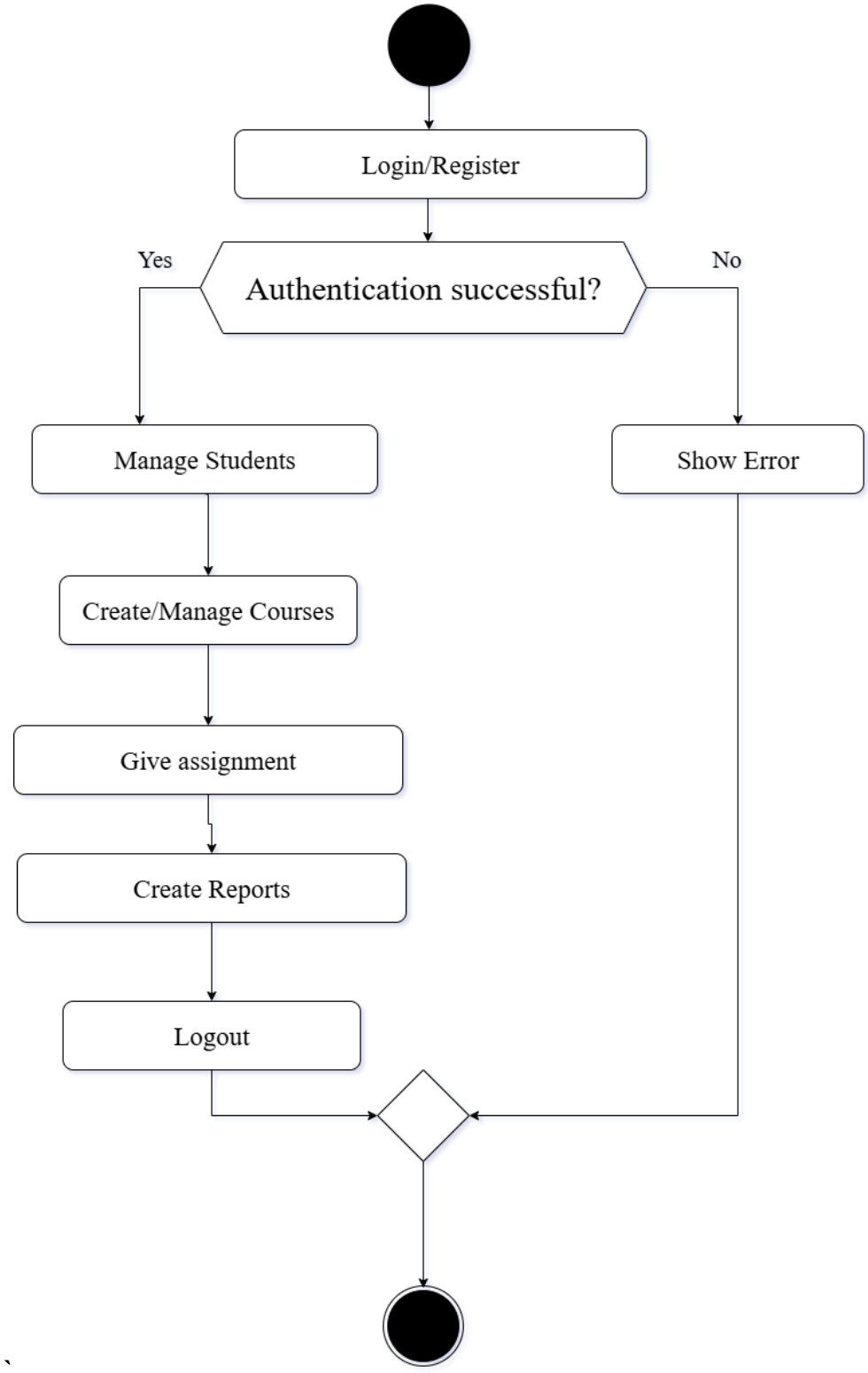


Figure 19: Process Diagram (Teacher)

Control flow Diagram

A control flow chart is a diagram to describe the control flow of the business process.

Admin:

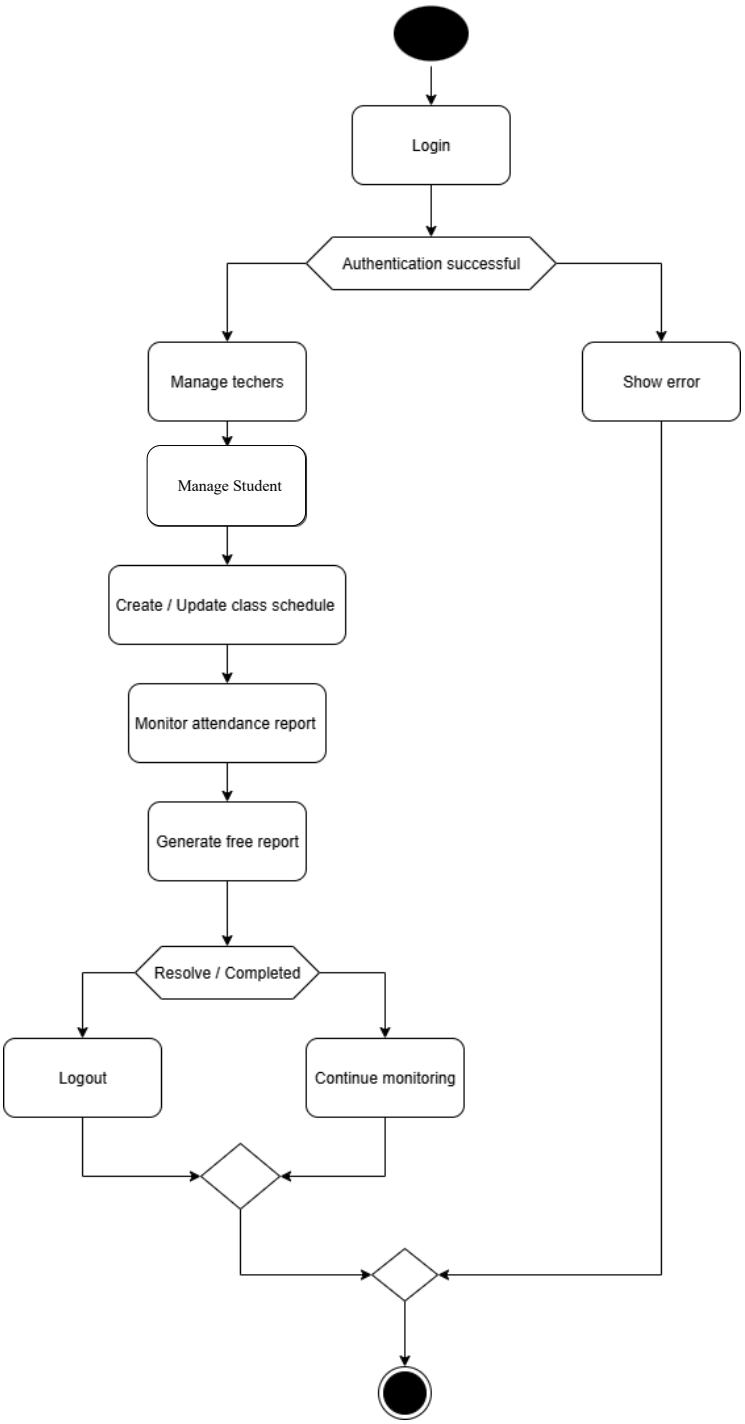


Figure 20:Control flow Diagram (Admin)

Student:

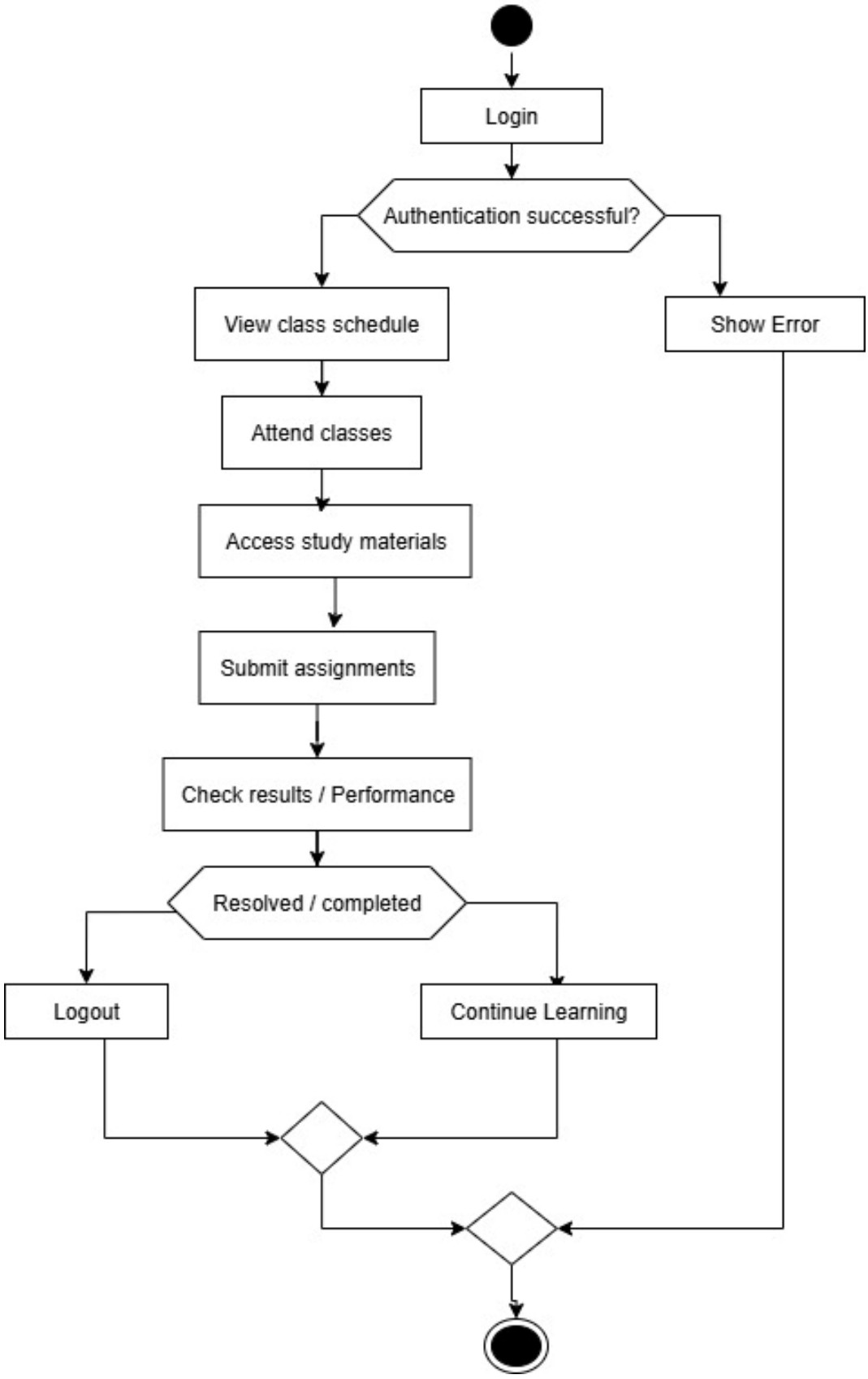


Figure 21:Control flow Diagram (Student)

Teacher:

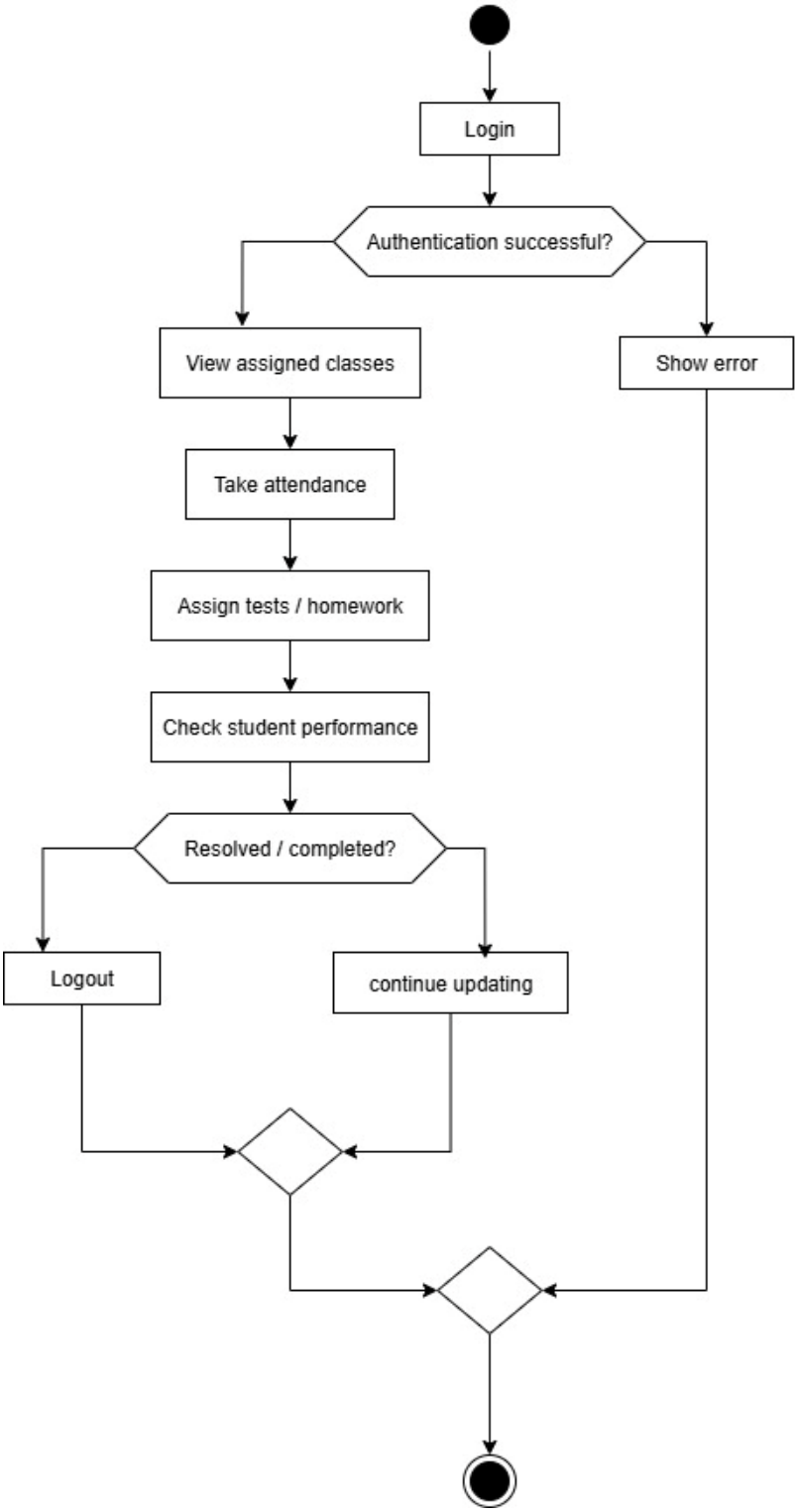


Figure 22:Control flow Diagram (Teacher)

4.3.2 Data Structures

The primary data structures used in procedures include:

- Arrays and HashMap for storing and retrieving user queries and responses.
- Linked Lists for maintaining appointment schedules.

1. Users Table (For Login/Authentication)

CREATE TABLE Users (

UserID INT PRIMARY KEY IDENTITY,

Username NVARCHAR (50) UNIQUE NOT NULL,

Password NVARCHAR (50) NOT NULL,

Role NVARCHAR (20) CHECK (Role IN ('Admin','Teacher','Clerk'))

);

2. Students Table

CREATE TABLE Students (

StudentID INT PRIMARY KEY IDENTITY,

Name NVARCHAR (100) NOT NULL,

Age INT,

Gender NVARCHAR (10),

ClassBatch NVARCHAR (50),

ContactNumber NVARCHAR (15),

Address NVARCHAR (255)

);

3. Teachers Table

CREATE TABLE Teachers (

TeacherID INT PRIMARY KEY IDENTITY,

Name NVARCHAR (100) NOT NULL,

```
Subject NVARCHAR (100),
Qualification NVARCHAR (100),
ContactNumber NVARCHAR (15),
Address NVARCHAR (255)
);
```

4. Receipts / Fees Table

```
CREATE TABLE Receipts (
ReceiptID INT PRIMARY KEY IDENTITY,
StudentID INT FOREIGN KEY REFERENCES Students (StudentID),
Amount DECIMAL (10,2) NOT NULL,
PaymentDate DATE NOT NULL,
PaymentMode NVARCHAR (20) CHECK (PaymentMode IN ('Cash','Card','UPI'))
);
```

4.3.3 Algorithms Design

1. Student Registration Algorithm

Steps:

1. Accept input: Name, Age, Gender, ClassBatch, Contact Number, Address.
2. Verify if the student already exists in the **student** table (check by Name + Contact Number).
3. If not, insert the record into the **student** table.
4. Generate a unique StudentID.
5. Create a Student Profile and return a success message.

2. User Login Algorithm (Admin/Teacher/Clerk)

Steps:

1. Accept input: Username, Password.
2. Retrieve the stored password hash from the **Users** table.
3. Hash the entered password and compare with stored password.
4. If they match → allow login and create a session.
5. Else → return **"Invalid Credentials"**.

3. Teacher Registration Algorithm

Steps:

1. Accept input: Name, Subject, Qualification, Contact Number, Address.
2. Validate all required fields.
3. Insert record into the **Teacher** table.
4. Generate a unique TeacherID.
5. Return success message **"Teacher Registered Successfully."**

4. Fee Receipt Generation Algorithm

Steps:

1. Accept input: StudentID, Amount, Payment Mode.
2. Validate if StudentID exists in the **student** table.
3. Generate a unique ReceiptID.
4. Insert record into the **Receipt** table with current date and time.
5. Print or Display receipt and return success message.

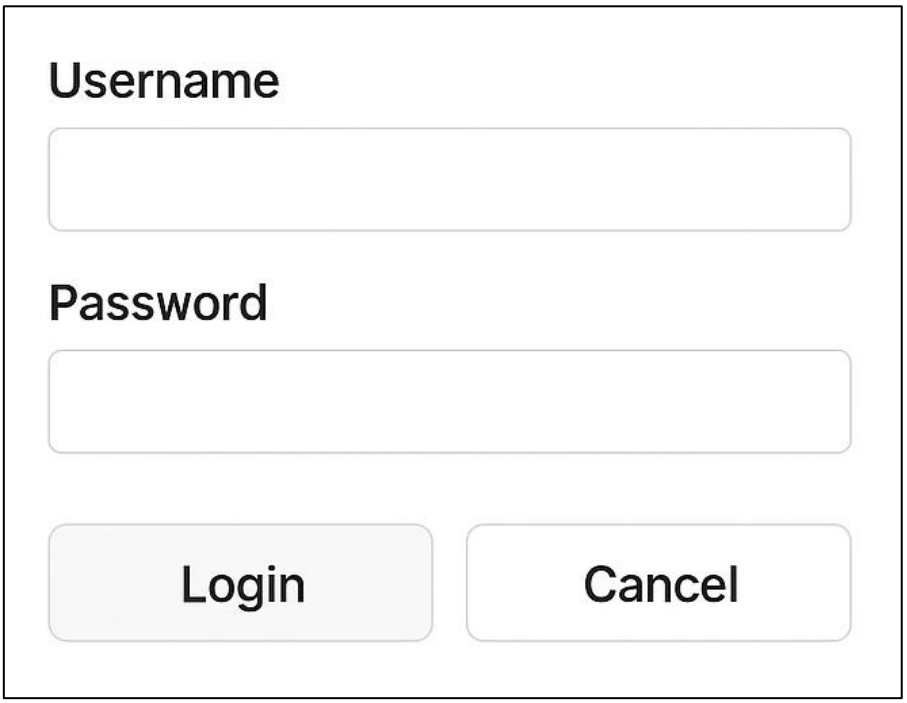
5. Reports Generation Algorithm

Steps:

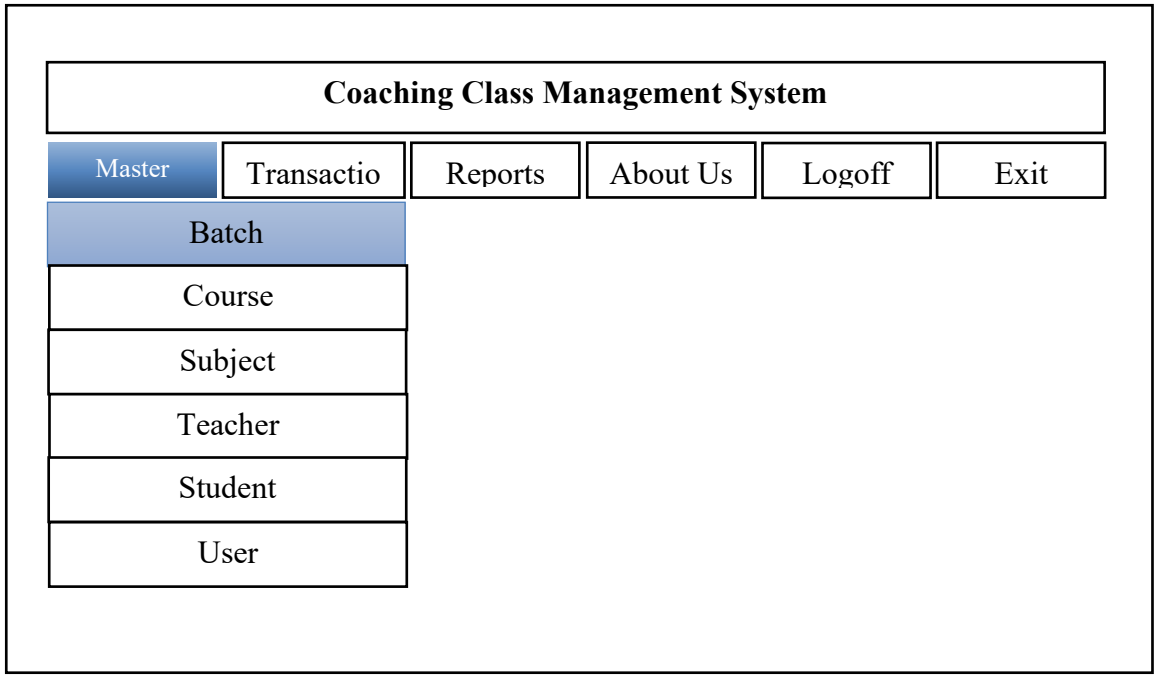
1. Accept input: Report Type (Student / Teacher / Fees).
2. If Student Report → Fetch all records from the **student** table.
3. If Teacher Report → Fetch all records from the **Teacher** table.
4. If Fees Report → Fetch records from the **Receipt** table joined with Student details.
5. Display results in tabular format.

4.4 User Interface Design

Authentication Form:



Main Screen:



4.5 Security Issues

Security is critical for handling sensitive healthcare data. The following security measures are implemented:

- **Data Encryption:** Sensitive data such as passwords and health records are encrypted, both in transit and at rest, to ensure data protection.
- **Authentication and Authorization:** OAuth and token-based authentication systems are employed to ensure that only authorized users can access the system. This provides a secure way of handling user identity and access control.
- **Data Privacy:** The system complies with data privacy regulations such as GDPR, ensuring that user data is handled securely, ethically, and with user consent. Access to personal health data is restricted and monitored.
- **NoSQL Injection Prevention:** Input validation and query parameterization are implemented to protect the MongoDB database from NoSQL injection attacks. By sanitizing inputs and avoiding raw queries, the system ensures that malicious data cannot compromise the database (OpenAI, 2025).

4.6 Test Cases Design

A test case has components that describe an input, action/event and an expected response, in order to determine if a feature of an application is working correctly. Test case is a set of instructions on "HOW" to validate a particular test objective/target, which when followed will tell us if the expected behaviour of the system is satisfied or not.

1. **User Login**

Step #	Step Details	Expected Results	Actual Results	Pass/Fail
1	Open login page	Login page is displayed		
2	Enter blank username & password	Error message: "Enter User Name & Password"		
3	Enter valid username & password	User is logged in and redirected to Home		
4	Enter invalid username/password	Error message: "Enter Correct User Name & Password"		

2. **Teacher Test Cases**

Step #	Step Details	Expected Results	Actual Results	Pass/Fail
1	Open Teacher module	Teacher dashboard is displayed		
2	Add Teacher with valid details	Teacher record added successfully		
3	Add Teacher with invalid details	Error message shown (e.g., "Invalid Input")		
4	Update existing Teacher details	Teacher record updated successfully		
5	Delete Teacher record	Teacher record deleted successfully		
6	View Teacher list	Teacher list displayed with all records		

3. Student Test Cases

Step #	Step Details	Expected Results	Actual Results	Pass/Fail
1	Open Student module	Student dashboard is displayed		
2	Add Student with valid details	Student record added successfully		
3	Add Student with invalid details	Error message shown (e.g., “Invalid Input”)		
4	Update Student details	Student record updated successfully		
5	Delete Student record	Student record deleted successfully		
6	View Student list	Student list displayed correctly		

4. Fee Test Cases

Step #	Step Details	Expected Results	Actual Results	Pass/Fail
1	Open Fee module	Fee dashboard is displayed		
2	Add Fee Record with valid details	Fee record added successfully		
3	Add Fee Record with invalid details	Error message displayed (e.g., “Invalid Fee Input”)		
4	Update Fee Record	Fee record updated successfully		
5	Delete Fee Record	Fee record deleted successfully		
6	View Fee details for student	Student fee details displayed correctly		

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 Implementation Approaches

Project implementation (or project execution) is the phase where visions and plans become reality. This is the logical conclusion, after evaluating, deciding, visioning, planning, applying for funds and finding the financial resources of a project. Technical implementation is one part of executing a project.

An implementation plan for a project refers to a detailed description of actions that demonstrate how to implement an activity within the project in the context of achieving project objectives, addressing requirements, and meeting expectations. (sswm.info, 2025)

A project team can take one of the three approaches for implementing the information system. These approaches include:

1. Direct cutover
2. Parallel
3. Phased

We use phased implementations for this project implementation. Parallel approach to implementation allows the old and the new systems to run concurrently for a time.

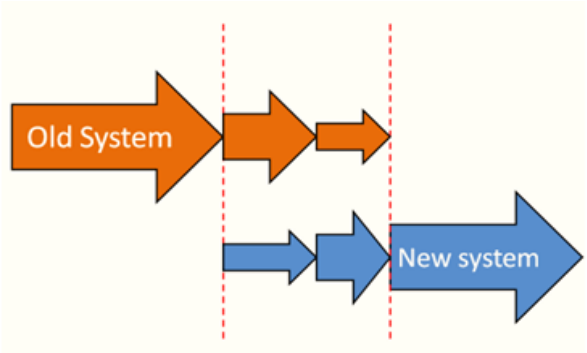


Figure 23: Phased Approach

In this approach we use some deliverables to implement project. These are Initial Phase, Project level installation phase, and maintenance phase.

5.2 Coding Details and Code Efficiency

5.2.1 Code Efficiency

Code efficiency is a broad term used to depict the reliability, speed and programming methodology used in developing codes for an application. Code efficiency is directly linked with algorithmic efficiency and the speed of runtime execution for software. It is the key element in ensuring high performance. The goal of code efficiency is to reduce resource consumption and completion time as much as possible with minimum risk to the business or operating environment. The software product quality can be accessed and evaluated with the help of the efficiency of the code used.

To improve performance of C# code we use following technique to optimize code.

- To remove unnecessary code or code that goes to redundant processing
- To make use of optimal memory and nonvolatile storage
- To ensure the best speed or run time for completing the algorithm
- To make use of reusable components wherever possible
- To make use of error and exception handling at all layers of software, such as the user interface, logic and data flow
- To create programming code that ensures data integrity and consistency
- To develop programming code that's compliant with the design logic and flow
- To make use of coding practices applicable to the related software
- To optimize the use of data access and data management practices
- To use the best keywords, data types and variables, and other available programming concepts to implement the related algorithm

5.3 Testing Approach

Implementation of test strategy for a particular project is known as "test approach". The test approach is usually defined in all test plans and test designs, i.e. how testing would be carried out.

Test approach has two techniques: **Proactive** - an approach in which the test design process is initiated as early as possible in order to find and fix the defects before

the build is created. **Reactive** - an approach in which the testing is not started until after design and coding are completed.

Functional Testing is defined as a type of testing which verifies that each function of the software application operates in conformance with the requirement specification. This testing mainly involves black box testing and it is not concerned about the source code of the application.

The prime objective of Functional testing is checking the functionalities of the software system. Examples of Functional testing are

- Unit Testing
- Smoke Testing
- Sanity Testing
- Integration Testing
- White box testing
- Black Box testing
- User Acceptance testing
- Regression Testing

5.3.1 Unit Testing

UNIT TESTING is a level of software testing where individual units/ components of a software are tested. The purpose is to validate that each unit of the software performs as designed. A unit is the smallest testable part of any software. It usually has one or a few inputs and usually a single output.

As a tester we could not wait till the entire application is developed and is made ready to test. In order to increase our time to market, we must start testing early.

We can execute all test cases, (positive and negative) to ensure that the Login form functionality is working as expected.

Advantages of login form testing is at this time are:

1. User Interface is tested for usability (spelling mistakes, logos, alignment, formatting etc.)
2. Trying negative testing techniques is a huge probability of finding defects.
3. To test the breach of security at a very early stage.

5.3.2 Integrated Testing

Integration testing ("I&T") is the phase in software testing in which individual software modules are combined and tested as a group. It occurs **after** unit testing and **before** system testing. Integration testing takes as its input modules that have been unit tested, groups them in larger aggregates, applies tests defined in an integration test plan to those aggregates, and delivers as its output the integrated system ready for system testing.

Integration testing scenario

User Action	Component	Testing Tools	Verification
Add, Update and Delete Student Data	Forms, Database	Visual Studio, SQL Server	Data should update correctly in database
User Login Validation	Login Form	C# Windows Forms	Only valid users can access dashboard
Save Student Record	Registration Module	SQL Server	Data stored and displayed in DataGridView
Record Attendance	Attendance Module	Database & UI	Attendance saved successfully

5.3.3 Beta Testing

Beta Testing is one of the Customer Validation methodologies to evaluate the level of customer satisfaction with the product by letting it to be validated by the end users, who actually use it, for over a period of time. Real People, Real Environment, Real Product are the three **R's** of Beta Testing.

5.4 Modifications and Improvements

Regression testing is a testing that is done to verify that a code change in the software does not impact the existing functionality of the product. Some issue occurs when checking user ID availability in the registration form and in order to fix the same, some code changes are done. In this case, not only the user ID need to be tested but

Acceptance User ID also needs to be tested to ensure that the change in the code has not affected them.

5.5 Test Cases

Test Case is a series of minimal simple steps that has to be done to check a particular functionality.

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Status
TC01	Check Login Functionality	Enter valid username & password and click login	Valid Credentials	Dashboard should open	Pass
TC02	Invalid Login Test	Enter wrong username/password	Invalid Data	Error message displayed	Pass
TC03	Add New Student	Enter all valid student details and click Save	Valid Student Data	Data successfully saved in database	Pass
TC04	Check Empty Field Validation	Leave required fields empty and click Save	Incomplete Data	Warning message shown	Pass
TC05	Update Student Record	Edit student details and click Update	Updated Data	Data updated in database	Pass
TC06	Delete Student Record	Select record and click Delete	Existing Record	Record deleted successfully	Pass
TC07	Attendance Entry	Mark attendance and save	Valid Attendance Data	Attendance stored in database	Pass
TC08	Fee Payment Entry	Enter fee details and save	Valid Fee Data	Fee record saved successfully	Pass

CHAPTER 6: RESULTS AND DISCUSSION

6.1 Test Report

Test Report is an important document that is prepared after the completion of the testing phase of the software project. The main objective of the test report is to present the testing activities, results, and overall system performance to stakeholders such as project guides, administrators, and end users.

For the **SheshCoachingClasses Management System**, comprehensive testing was conducted on all major modules including Login, Student Management, Attendance, Fee Management, and Report Generation. Both functional and integration testing techniques were used to verify the correctness, reliability, and efficiency of the system.

After executing all the test cases, the system showed stable performance with successful execution of most functionalities. Minor issues related to validation and UI alignment were identified and corrected during the debugging phase.

Test Report Summary

Result	Count
Passed	20
Failed	2

The majority of the test cases passed successfully, indicating that the system meets the required functional specifications and performs efficiently in a real-time coaching class environment.

Module-wise Test Report

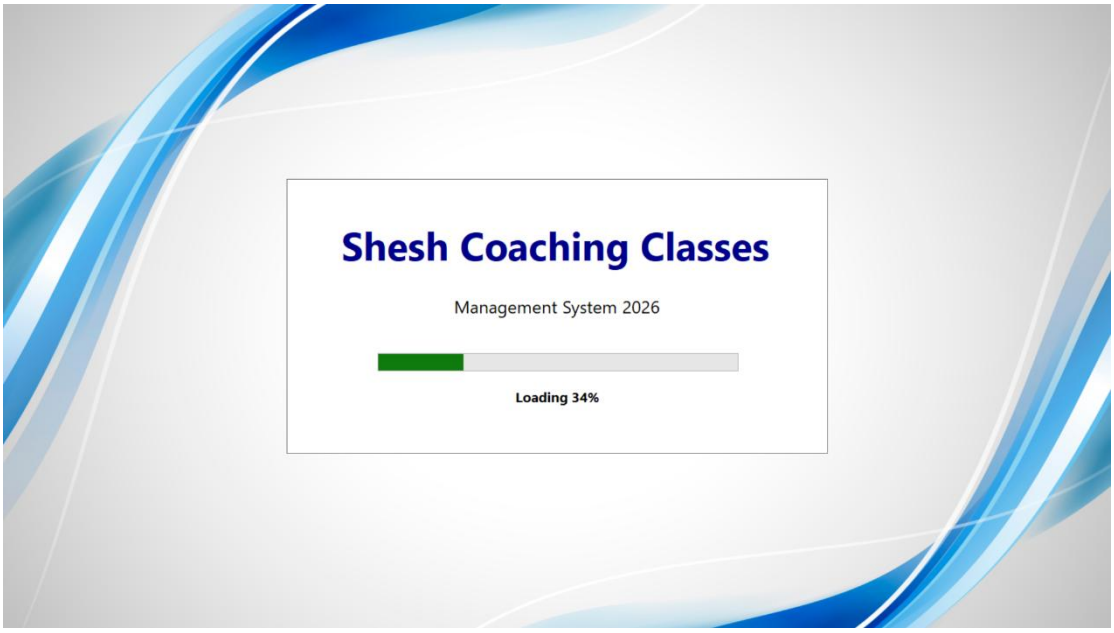
Functions	Description	% TCs Executed	% TCs Passed	TCs Pending	Priority
New User Registration	Check new student record is created	100%	100%	0	High
Edit Student	Check student details can be updated	100%	100%	0	High
Delete Student Record	Verify student record deletion	100%	100%	0	High
Login Module	Check dashboard opens after valid login	100%	100%	0	High
Student Entry	Check new student data is added	100%	100%	0	High

Update Student Data	Check student details can be updated correctly	100%	100%	0	High
Attendance Module	Check attendance is recorded and saved	100%	100%	0	High
Fee Management	Check fee details are stored and displayed	100%	100%	0	High
Generate Reports	Check student list and fee reports generation	100%	100%	0	High
Database Backup	Check backup and restore functionality	100%	0%	0	High

6.2 User Documentation

User documentation is important in any development project. These documents help to explain our product to users by providing them with necessary information. The user manual is essential for every software product because it serves as the ultimate guide regarding our product.

Splash Screen

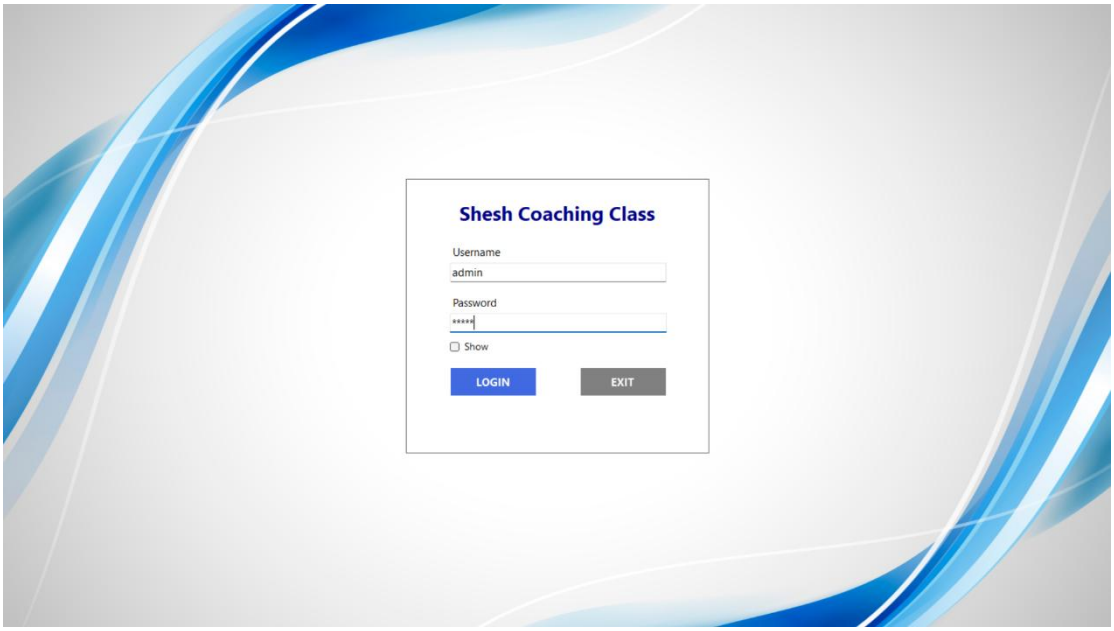


Splash.cs

```
using System;
using System.Drawing;
using System.IO;
using System.Windows.Forms;

namespace SheshCoachingCalsses
{
   public partial class splash : Form
   {
      public splash()
      {
         InitializeComponent();
         // FULL SCREEN
         this.WindowState = FormWindowState.Maximized;
         this.FormBorderStyle = FormBorderStyle.None;
         this.DoubleBuffered = true;
      }
      private void splash_Load(object sender, EventArgs e)
      {
         // SAFE BACKGROUND IMAGE LOAD
         string img = @"C:\Users\shesh\Desktop\rm222batch5-kul-18.jpg";
         if (File.Exists(img))
         {
            this.BackgroundImage = Image.FromFile(img);
            this.BackgroundImageLayout = ImageLayout.Stretch;
         }
         else
         {
            this.BackColor = Color.LightBlue;
         }
```

```
            CenterPanel();
        }
        // CENTER PANEL
        private void CenterPanel()
        {
            panel1.Left = (this.ClientSize.Width - panel1.Width) / 2;
            panel1.Top = (this.ClientSize.Height - panel1.Height) / 2;
        }
        protected override void OnResize(EventArgs e)
        {
            base.OnResize(e);
            CenterPanel();
        }
        private void Timer1_Tick(object sender, EventArgs e)
        {
            if (pgbar.Value < 100)
            {
                pgbar.Value += 2;
                labelLoading.Text = "Loading " + pgbar.Value + "%";
            }
            else
            {
                Timer1.Stop();
                this.Hide();

                login lg = new login();
                lg.Show();
            }
        }
    }
}
```

Login Screen



Login.cs

```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.IO;
using System.Windows.Forms;

namespace SheshCoachingCalsses
{
   public partial class login : Form
   {
      public login()
      {
         InitializeComponent();

         this.WindowState = FormWindowState.Maximized;
         this.FormBorderStyle = FormBorderStyle.None;
         this.DoubleBuffered = true;
      }
```

```
        private void login_Load(object sender, EventArgs e)
        {
            string img = @"C:\Users\shesh\Desktop\rm222batch5-kul-18.jpg";
            if (File.Exists(img))
            {
                this.BackgroundImage = Image.FromFile(img);
                this.BackgroundImageLayout = ImageLayout.Stretch;
            }
            CenterPanel();
        }
        // CENTER PANEL
        private void CenterPanel()
        {
            panel1.Left = (this.ClientSize.Width - panel1.Width) / 2;
            panel1.Top = (this.ClientSize.Height - panel1.Height) / 2;
        }
        protected override void OnResize(EventArgs e)
        {
            base.OnResize(e);
            CenterPanel();
        }
        // LOGIN BUTTON
        private void btnLogin_Click(object sender, EventArgs e)
        {
            string uname = txtname.Text;
            string pass = txtpass.Text;
            SqlConnection con = new SqlConnection(
            "Data Source=.\\SQLEXPRESS;Initial Catalog=mclass;Integrated
Security=True");
            con.Open();
```

```
            txtpass.Clear();
            txtname.Focus();
        }
        con.Close();
    }
    // EXIT BUTTON
    private void btnClose_Click(object sender, EventArgs e)
    {
        Application.Exit();
    }
    // SHOW PASSWORD
    private void chkShowPassword_CheckedChanged(object sender, EventArgs e)
    {
        if (chkShowPassword.Checked)
            txtpass.PasswordChar = '\0';
        else
            txtpass.PasswordChar = '*';
    }
  }
}
```

Main Screen

Main.cs

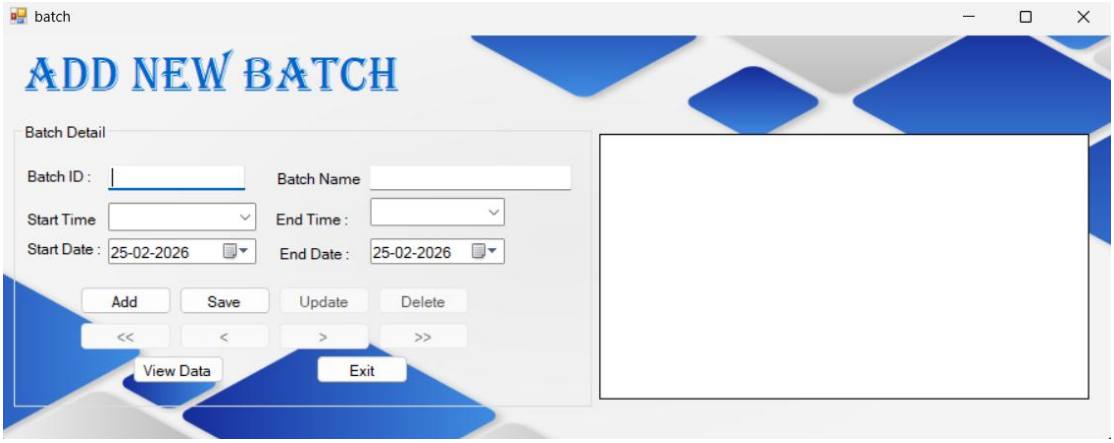
```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;

namespace SheshCoachingCalsses
{
   public partial class Main : Form
   {
      public Main()
      {
         InitializeComponent();
      }
      private void exitToolStripMenuItem_Click(object sender, EventArgs e)
      {
         Application.Exit();
      }
      private void batchToolStripMenuItem_Click(object sender, EventArgs e)
      {
         new batch().Show();
      }
      private void courseToolStripMenuItem_Click(object sender, EventArgs e)
      {
         new course().Show();
      }
      private void subjectToolStripMenuItem_Click(object sender, EventArgs e)
      {
```

```
        {
            new tlist().Show();
        }
        private void courseListToolStripMenuItem_Click(object sender, EventArgs e)
        {
            new clist().Show();
        }
        private void paymentReportToolStripMenuItem_Click(object sender, EventArgs
e)
        {
            new payrpt().Show();
        }


        private void feeReportToolStripMenuItem_Click(object sender, EventArgs e)
        {
            new feerpt().Show();
        }
        private void aboutUsToolStripMenuItem_Click(object sender, EventArgs e)
        {
            new AboutBox1().Show();
        }
        private void logoffToolStripMenuItem_Click(object sender, EventArgs e)
        {
            new login().Show();
            this.Hide();
        }
        private void attendanceReportToolStripMenuItem_Click(object sender,
EventArgs e)
        {
            new attendrpt().Show();
        }
        private void userToolStripMenuItem_Click(object sender, EventArgs e)
        {
```

```
            new umanage().Show();
        }
    }
}
```

1.Master

Batch



Batch.cs

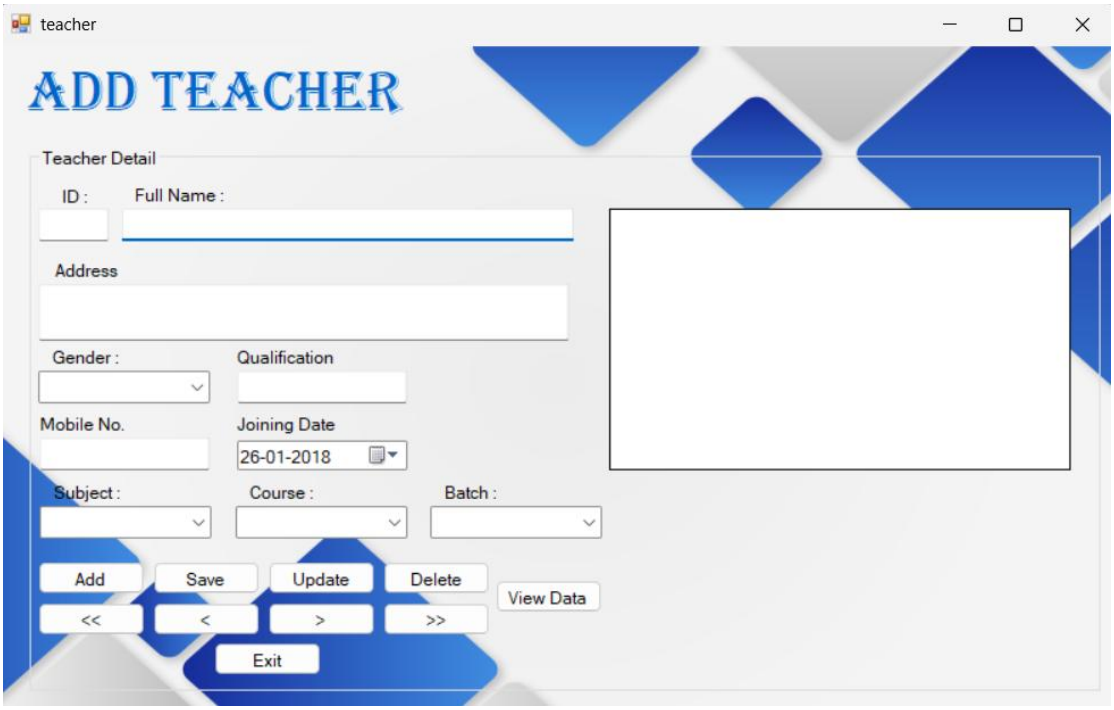
```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Windows.Forms;


namespace SheshCoachingCalsses
{
    public partial class batch : Form
    {
        private readonly SqlConnection con =
     new SqlConnection(@"Data Source=.\sqlexpress;Initial Catalog=mclass;Integrated Security=True");
        private readonly DataSet ds = new DataSet();
        int i = 0;
        public batch()
        {
            InitializeComponent();
```

```
        }
        // ================== ADD ================
        private void BtnAdd_Click(object sender, EventArgs e)
        {
            try
            {
                con.Open();
                SqlCommand cmd = new SqlCommand("SELECT
ISNULL(MAX(bid),0)+1 FROM batch", con);
                txtbid.Text = cmd.ExecuteScalar().ToString();
                con.Close();

                btnSave.Enabled = true;
                btnAdd.Enabled = false;
                dgcourse.Enabled = false;
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message);
            }
        }
        // ================== LOAD DATA ================
        private void Loadlist()
        {
            try
            {
                ds.Clear();
                SqlDataAdapter da = new SqlDataAdapter("SELECT * FROM batch",
con);
                da.Fill(ds);
                dgcourse.DataSource = ds.Tables[0];
            }
            catch (Exception ex)
```

```
private void BtnNext_Click(object sender, EventArgs e)
{
   if (i < ds.Tables[0].Rows.Count - 1)
   {
      i++;
      ShowRecord();
   }
}
private void BtnPrevious_Click(object sender, EventArgs e)
{
   if (i > 0)
   {
      i--;
      ShowRecord();
   }
}
private void ShowRecord()
{
   txtbid.Text = ds.Tables[0].Rows[i]["bid"].ToString();
   txtbname.Text = ds.Tables[0].Rows[i]["bname"].ToString();
   dtps.Value = Convert.ToDateTime(ds.Tables[0].Rows[i]["sdt"]);
   dtpe.Value = Convert.ToDateTime(ds.Tables[0].Rows[i]["edt"]);
   stcmb1.Text = ds.Tables[0].Rows[i]["stime"].ToString();
   etcmb2.Text = ds.Tables[0].Rows[i]["etime"].ToString();
}
private void Button1_Click(object sender, EventArgs e)
{
   Loadlist();
}
private void BtnExit_Click(object sender, EventArgs e)
```

```
        {
            this.Close();
        }
    }
}
```

Teacher



Teacher.cs

```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Windows.Forms;


namespace SheshCoachingCalsses
{
    public partial class teacher : Form
    {
        SqlConnection con = new SqlConnection(@"Data Source=.\sqlexpress;Initial Catalog=mclass;Integrated Security=True");
        DataSet ds = new DataSet();
        int i = 0;
```

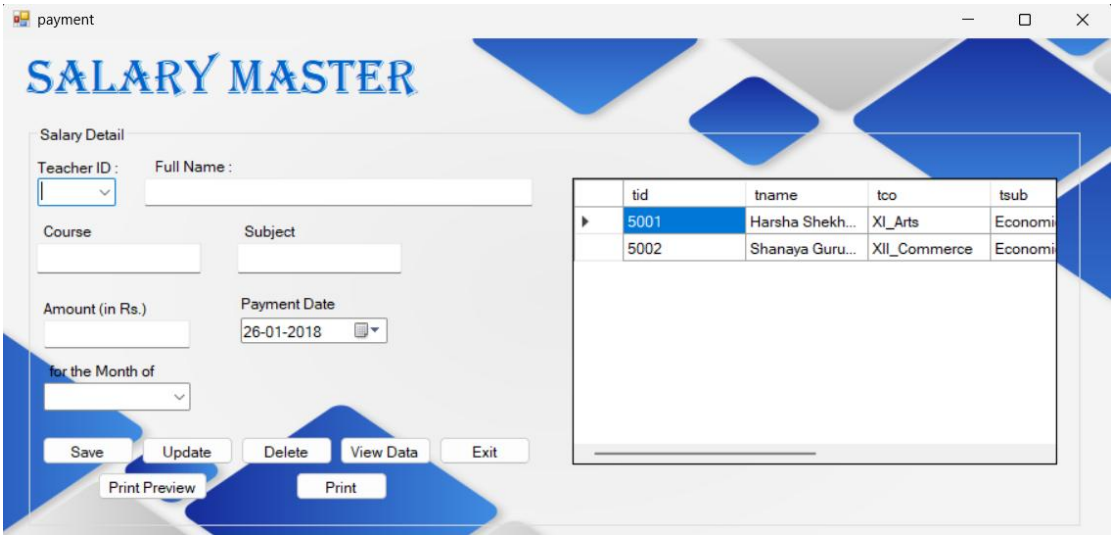
```
public teacher()
{
   InitializeComponent();
   LoadComboData();
}
private void LoadComboData()
{
   try
   {
      con.Open();
      // Course
      SqlCommand cmd = new SqlCommand("select cname from course", con);
      SqlDataReader dr = cmd.ExecuteReader();
      while (dr.Read())
      {
         cmbco.Items.Add(dr[0].ToString());
      }
      dr.Close();
      // Subject
      cmd = new SqlCommand("select sname from subject", con);
      dr = cmd.ExecuteReader();
      while (dr.Read())
      {
         cmbsub.Items.Add(dr[0].ToString());
      }
      dr.Close();
      // Batch
      cmd = new SqlCommand("select bname from batch", con);
      dr = cmd.ExecuteReader();
      while (dr.Read())
      {
```

```
    }
    private void clear()
    {
        txtid.Clear();
        txtname.Clear();
        txtadd.Clear();
        txtmobi.Clear();
        txtquali.Clear();
        cmbgen.SelectedIndex = -1;
        cmbsub.SelectedIndex = -1;
        cmbco.SelectedIndex = -1;
        cmbbat.SelectedIndex = -1;
        dtp1.Value = DateTime.Now; // FIXED DATE CLEAR
    }
    private void button1_Click(object sender, EventArgs e)
    {
        loadlist();
    }
    private void dgcourse_CellClick(object sender, DataGridViewCellEventArgs e)
    {
        if (e.RowIndex >= 0)
        {
            DataGridViewRow row = dgcourse.Rows[e.RowIndex];
            txtid.Text = row.Cells[0].Value.ToString();
            txtname.Text = row.Cells[1].Value.ToString();
            txtadd.Text = row.Cells[2].Value.ToString();
            cmbgen.Text = row.Cells[3].Value.ToString();
            txtquali.Text = row.Cells[4].Value.ToString();
            txtmobi.Text = row.Cells[5].Value.ToString();
            dtp1.Value = Convert.ToDateTime(row.Cells[6].Value); // FIXED DATE
LOAD
            cmbsub.Text = row.Cells[7].Value.ToString();
```

```
            cmbco.Text = row.Cells[8].Value.ToString();
            cmbbat.Text = row.Cells[9].Value.ToString();
        }
    }
  }
}
```

2.Transaction

Payment



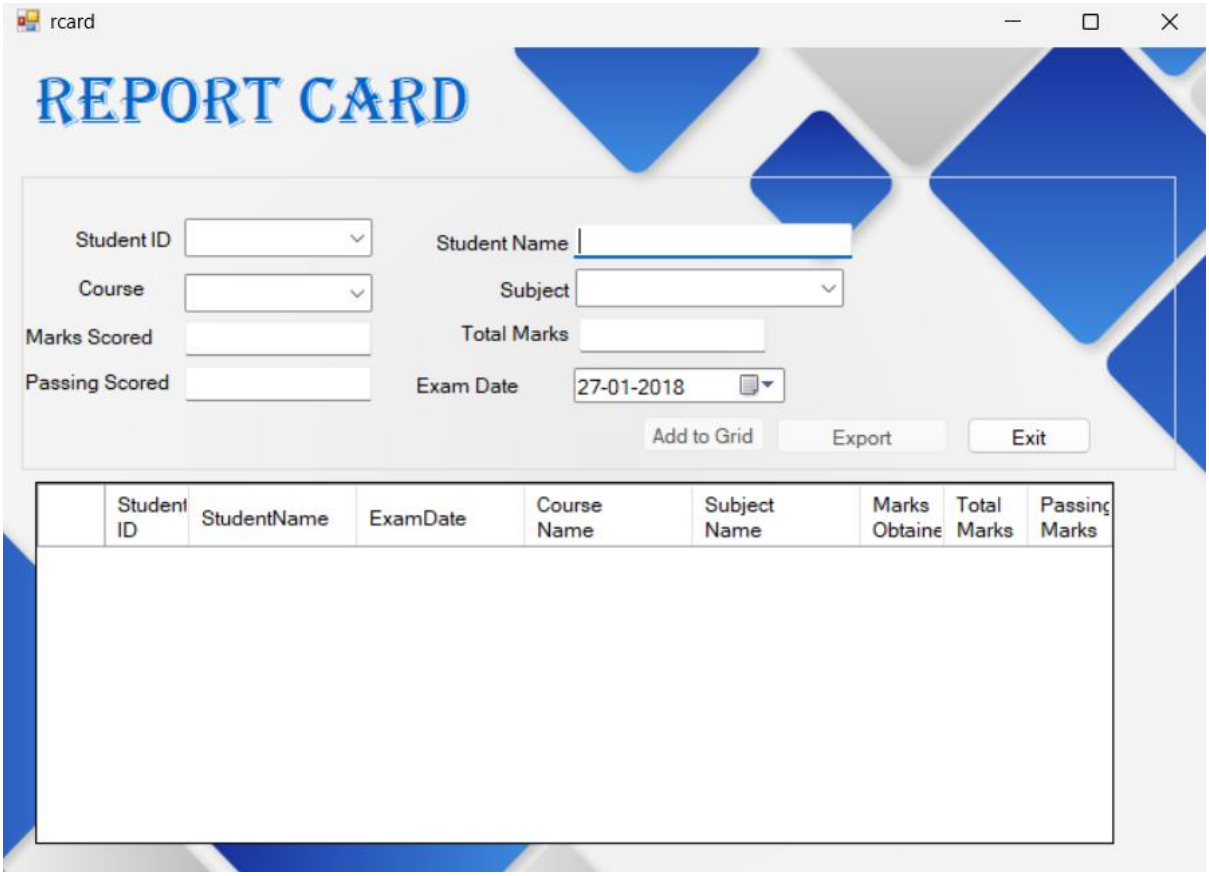
Payment.cs

```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Drawing;
using System.Windows.Forms;

namespace SheshCoachingCalsses
{
   public partial class payment : Form
   {
      SqlConnection con = new SqlConnection(@"Data Source=.\sqlexpress;Initial
Catalog=mclass;Integrated Security=True");
      DataSet ds = new DataSet();
```

```
        public payment()
        {
            InitializeComponent();
            LoadTeacherIds();
            LoadList();
        }
        private void LoadTeacherIds()
        {
            try
            {
                cmbtid.Items.Clear();
                con.Open();
                SqlCommand cmd = new SqlCommand("SELECT tid FROM teacher",
con);
                SqlDataReader dr = cmd.ExecuteReader();
                while (dr.Read())
                {
                    cmbtid.Items.Add(dr["tid"].ToString());
                }
                dr.Close();
                con.Close();
                cmbtid.Text = row.Cells[0].Value.ToString();
                txtname.Text = row.Cells[1].Value.ToString();
                txtco.Text = row.Cells[2].Value.ToString();
                txtsub.Text = row.Cells[3].Value.ToString();
                txtmobi.Text = row.Cells[4].Value.ToString();
                dtp1.Value = Convert.ToDateTime(row.Cells[5].Value);
                comboBox1.Text = row.Cells[6].Value.ToString();
            }
        }
        private void btnExit_Click(object sender, EventArgs e)
        {
```

```
            this.Close();
        }
        private void button3_Click(object sender, EventArgs e)
        {
            printPreviewDialog1.Document = printDocument1;
            printPreviewDialog1.ShowDialog();
        }
        private void button2_Click(object sender, EventArgs e)
        {
            printDocument1.Print();
        }
        private void printDocument1_PrintPage(object sender,
System.Drawing.Printing.PrintPageEventArgs e)
        {
            Font title = new Font("Book Antiqua", 22, FontStyle.Bold);
            Font body = new Font("Times New Roman", 12);
            e.Graphics.DrawString("Shesh Bhoir : Place to Learn", title,
Brushes.Chocolate, 150, 40);
            e.Graphics.DrawString("Receipt", title, Brushes.Black, 300, 90);
            e.Graphics.DrawString("Teacher ID: " + cmbtid.Text, body, Brushes.Black,
100, 150);
            e.Graphics.DrawString("Teacher Name: " + txtname.Text, body,
Brushes.Black, 100, 180);
            e.Graphics.DrawString("Course: " + txtco.Text, body, Brushes.Black, 100,
210);
            e.Graphics.DrawString("Subject: " + txtsub.Text, body, Brushes.Black, 100,
240);
            e.Graphics.DrawString("Amount: Rs. " + txtmobi.Text, body, Brushes.Black,
100, 270);
            e.Graphics.DrawString("Payment Date: " + dtp1.Value.ToShortDateString(),
body, Brushes.Black, 100, 300);
            e.Graphics.DrawString("Month: " + comboBox1.Text, body, Brushes.Black,
100, 330);
            e.Graphics.DrawString("Signature", body, Brushes.Black, 450, 380);
        }
```

Rcard



Rcard.cs

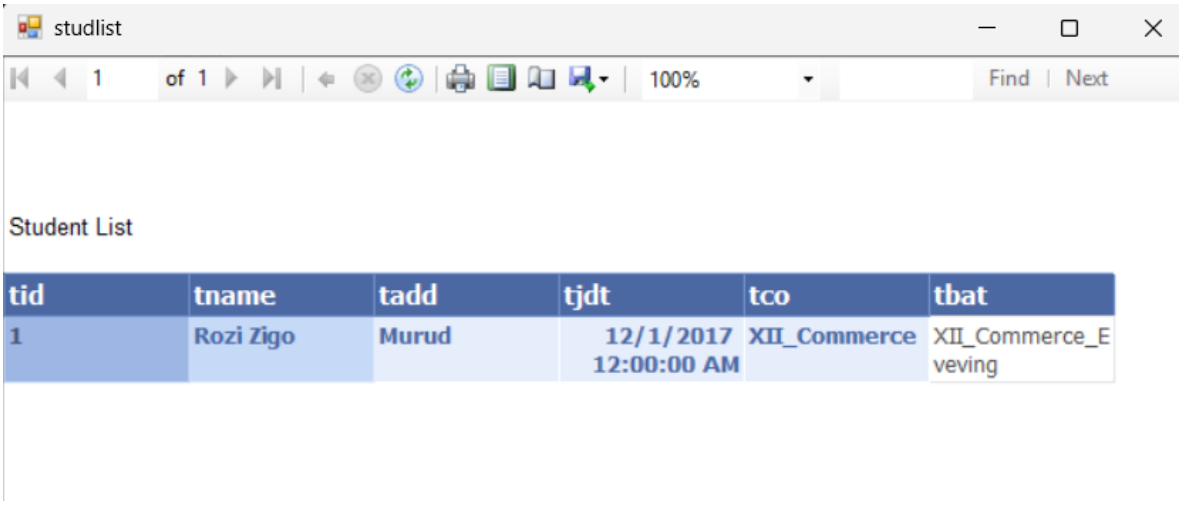
```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Windows.Forms;
using Excel = Microsoft.Office.Interop.Excel;


namespace SheshCoachingCalsses
{
   public partial class rcard : Form
   {
      SqlConnection con = new SqlConnection(@"Data Source=.\sqlexpress;Initial
Catalog=mclass;Integrated Security=True");
      public rcard()
      {
```

```
            InitializeComponent();
            LoadData();
        }
        private void LoadData()
        {
            try
            {
                con.Open();
                // Load Student IDs
                SqlCommand cmd1 = new SqlCommand("SELECT tid FROM student",
con);
                SqlDataReader dr1 = cmd1.ExecuteReader();
                cmbsid.Items.Clear();
                while (dr1.Read())
                {
                    cmbsid.Items.Add(dr1["tid"].ToString());
                }
            {
                Excel.Application xlApp = new Excel.Application();
                Excel.Workbook xlWorkBook = xlApp.Workbooks.Add(Type.Missing);
                Excel.Worksheet xlWorkSheet = (Excel.Worksheet)xlWorkBook.Sheets[1];
                for (int i = 0; i < dgv.Rows.Count; i++)
                {
                    for (int j = 0; j < dgv.Columns.Count; j++)
                    {
                        xlWorkSheet.Cells[i + 1, j + 1] =
dgv.Rows[i].Cells[j].Value?.ToString();
                    }
                }
                string path =
Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments) +
"\\ReportCard.xls";
                xlWorkBook.SaveAs(path, Excel.XlFileFormat.xlWorkbookNormal);
```

```
            xlWorkBook.Close(true);
            xlApp.Quit();
            MessageBox.Show("Excel file created in My Documents", "Export
Success");
        }
        catch (Exception ex)
        {
            MessageBox.Show(ex.Message, "Export Error");
        }
    }
    private void ClearFields()
    {
        cmbsid.Text = "";
        txtbname.Text = "";
        cmbco.Text = "";
        cmbsub.Text = "";
        txtms.Text = "";
        txttm.Text = "";
        txtps.Text = "";
        dtp.Value = DateTime.Now;
    }
    private void BtnExit_Click(object sender, EventArgs e)
    {
        this.Close();
    }
    private void Dtp_ValueChanged_1(object sender, EventArgs e)
    {
        btnAdd.Enabled = true;
    }
  }
}
```

3.Reports

Student List

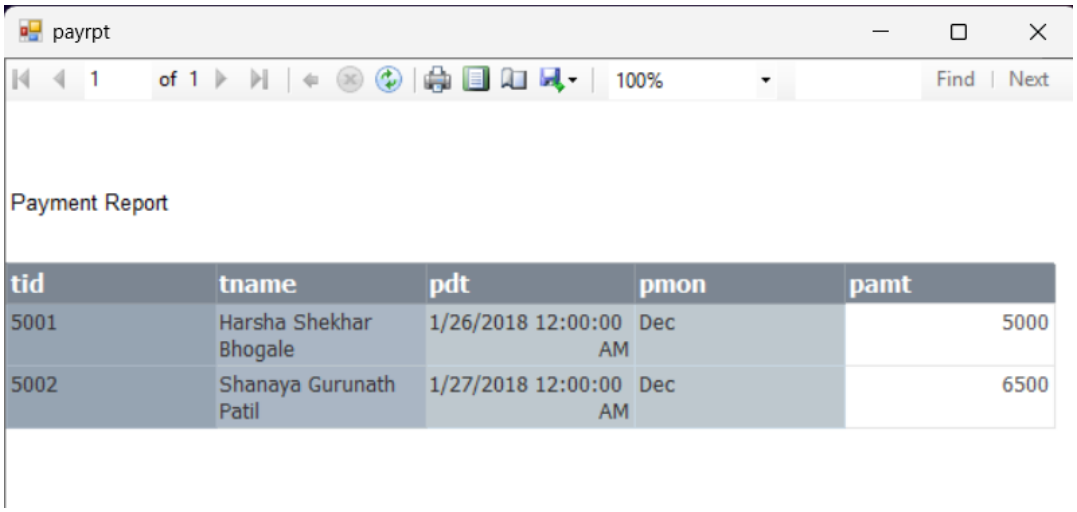


StudentList.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;

namespace SheshCoachingCalsses
{
   public partial class studlist : Form
   {
      public studlist()
      {
         InitializeComponent();
      }
      private void studlist_Load(object sender, EventArgs e)
      {
```

```
            // TODO: This line of code loads data into the 'mclassDataSet.student' table.
You can move, or remove it, as needed.
            this.studentTableAdapter.Fill(this.mclassDataSet.student);


            this.reportViewer1.RefreshReport();
        }
    }
}
```

Payment Report



Payrpt.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;

namespace SheshCoachingCalsses
{
    public partial class payrpt : Form
    {
```

```
        public payrpt()
        {
            InitializeComponent();
        }
        private void payrpt_Load(object sender, EventArgs e)
        {
            // TODO: This line of code loads data into the 'mclassDataSet3.payment' table.
You can move, or remove it, as needed.
            this.paymentTableAdapter.Fill(this.mclassDataSet3.payment);
            this.reportViewer1.RefreshReport();
        }
    }
}
```

CHAPTER 7: CONCLUSION

7.1 Conclusion

The Shesh Coaching Classes Management System is developed to simplify and automate the daily operations of coaching institutes. The system provides an efficient digital platform to manage student records, fees, batches, subjects, attendance, and other administrative activities. This software helps reduce manual work, minimize errors, and improve overall efficiency in managing coaching classes.

The main objective of the project is to provide a centralized and user-friendly system that supports smooth academic and administrative functioning. The system enables better data organization, quick access to information, and improved communication between administration and students.

This project has been developed after studying current technological trends and user requirements of coaching institutes. The system is designed to be practical, easy to use, and adaptable to real-world educational environments.

Finally, it is concluded that the project successfully achieves the following:

- Provided a detailed understanding of the coaching institute management process.
- Clearly defined the aims and objectives of the system.
- Identified the problem areas in manual management and proposed digital solutions.
- Designed and implemented modules for student management, fee tracking, batch allocation, and attendance.
- Developed a structured database to maintain records efficiently.
- Created a user-friendly interface for easy operation.
- Ensured proper data handling and basic security features.
- Successfully implemented and tested the system according to required test cases.

7.1.1 Significance of System

- The system helps coaching institutes manage student and academic information in an organized and systematic way.
- It reduces paperwork and manual record-keeping, saving time and effort.
- It improves accuracy in fee management, attendance tracking, and batch scheduling.
- It provides quick access to information, helping administrators make better decisions.
- It enhances the overall productivity and efficiency of coaching institute operations.

7.2 Limitations of the System

- The system currently supports limited report generation features.
- It is designed for basic administrative use and may require enhancements for large-scale institutions.
- Advanced analytics and performance tracking features are not included.
- The system requires manual data entry for certain operations.
- Online access and cloud integration are not implemented in the current version.

7.3 Future Scope of the Project

- Integration of real-time attendance tracking and automated notifications.
- Development of advanced reporting and performance analysis tools.
- Online student portal for registration, fee payment, and progress tracking.
- Mobile application (Android/iOS) for easy access.
- Cloud-based database for remote access and data backup.
- SMS or email notification system for announcements and reminders.

These enhancements will make the system more powerful, scalable, and suitable for modern digital learning environments.

Finally, we express our sincere gratitude to all the individuals who directly or indirectly supported the development of this project.

Bibliography

(2025, 08 20). Retrieved from OpenAI: https://chat.openai.com/

(2025, 08 20). Retrieved from OpenAI: https://chat.openai.com/

(2025, 08 20). Retrieved from OpenAI: https://chat.openai.com/

Class Diagram | Unified Modeling Language (UML). (2025, 08 19). Retrieved from GeeksforGeeks: https://www.geeksforgeeks.org/unified-modeling-language-uml-class-diagrams/

Flatirons. (2025, 8 9). Retrieved from Flatirons: https://flatirons.com/blog/what-is-reactjs/

IBM. (2025, 08 20). Retrieved from IBM: https://www.ibm.com/docs/en/rsm/7.5.0?topic=uml-sequence-diagrams

IBM. (2025, 08 20). Retrieved from IBM: https://www.ibm.com/docs/en/rational-soft-arch/9.7.0?topic=diagrams-deployment

Integrate.io. (2025, 08 20). Retrieved from Integrate.io: https://www.integrate.io/blog/complete-guide-to-database-schema-design-guide/

Lucidchart. (2025, 08 19). Retrieved from Lucidchart: https://www.lucidchart.com/pages/er-diagrams

MDN Web Docs. (2025, 08 09). Retrieved from Mozilla: https://developer.mozilla.org/en-US/docs/Web/JavaScript

Microsoft. (2025, 08 09). Retrieved from Microsoft: https://docs.microsoft.com/en-us/dotnet/framework/get-started/overview

MindManager. (2025, 08 20). Retrieved from MindManager: https://www.mindmanager.com/en/features/activity-diagram/#:~:text=An%20activity%20diagram%20is%20a,happen%20in%20the%20modeled%20system.

miro. (2025, 08 20). Retrieved from miro: https://miro.com/diagramming/why-is-database-design-important/

Modern Systems Analysis and Design. (2025, 8 2). Retrieved from geeksforgeeks: https://www.geeksforgeeks.org/top-8-software-development-models-used-in-industry/

MongoDB. (2025, 08 16). Retrieved from MongoDB: https://www.tutorialspoint.com/mongodb/mongodb_overview.htm

MySQL. (2025, 08 09). Retrieved from Wikipedia: https://en.wikipedia.org/wiki/MySQL

OpenAI. (2025, 8 18). *Problem Definetion and Requirement Specification*. Retrieved from OpenAI: https://chat.openai.com/

OpenAI. (2025, 8 09). *Survey of Tech*. Retrieved from OpenAI: https://chat.openai.com/

Oracle Database. (2025, 8 9). Retrieved from Wikipedia: https://en.wikipedia.org/wiki/Oracle_Database

Python.org. (2025, 8 9). Retrieved from Python.org: https://www.python.org/doc/essays/blurb/

smartdraw. (2025, 08 20). Retrieved from smartdraw: https://www.smartdraw.com/component-diagram/#:~:text=A%20component%20diagram%2C%20also%20known,is%20covered%20by%20planned%20development.

Techtarget. (2025, 08 09). Retrieved from Techtarget: https://www.techtarget.com/searchdatamanagement/definition/database-management-system

Tutorial Point. (2025, 08 09). Retrieved from Tutorial Point: https://www.tutorialspoint.com/cplusplus/cpp_overview.htm

Tutorial Point. (2025, 08 20). Retrieved from Tutorial Point: https://www.tutorialspoint.com/ms_sql_server/index.htm

Tutorial Point. (2025, 08 9). Retrieved from Tutorial Point: https://www.tutorialspoint.com/python/python_overview.htm

Tutorial Point. (2025, 8 9). Retrieved from Tutorial Point: https://www.tutorialspoint.com/postgresql/

Tutorialspoint. (2025, 08 20). Retrieved from Tutorialspoint: https://www.tutorialspoint.com/uml/uml_statechart_diagram.htm

w3resource. (2025, 09 8). Retrieved from w3resource: https://www.w3resource.com/csharp-exercises/

W3Schools. (2025, 8 9). Retrieved from W3Schools: https://www.w3schools.com/css/css_intro.asp

Wikipedia. (2025, 08 19). Retrieved from Wikipedia: https://en.wikipedia.org/wiki/Use_case_diagram#:~:text=A%20use%20case%20diagram%20is,by%20either%20circles%20or%20ellipses.

Wikipedia. (2025, 08 09). Retrieved from Wikipedia: https://en.wikipedia.org/wiki/Java_(programming_language)